VELLORE INSTITUTE OF TECHNOLOGY, VELLORE

School of Computer Science and Engineering

B.Tech. Computer Science and Engineering

FINAL PROJECT REPORT – REVIEW 3

BCSE307L – Compiler Design

Range-Aware Edge AI Quantization Compiler

with the RANGE//LAB INT8 Quantization Workbench

Submitted by

Yash Pradhan	24BCE0702
Anjini Pandey	24BCE0714
Ishita Srivastava	24BDS0234

Faculty Guide

Prof. Kanagaraj R.

October 2026

TABLE OF CONTENTS

Sl.No	Contents	Page No.
1.	**INTRODUCTION**	2
	1.1 BACKGROUND	2
	1.2 MOTIVATIONS	2
	1.3 SCOPE OF THE PROJECT	3
2.	**PROJECT DESCRIPTION AND GOALS**	4
	2.1 LITERATURE REVIEW	4
	2.2 RESEARCH GAP	6
	2.3 OBJECTIVES	7
	2.4 PROBLEM STATEMENT	8
4.	**DESIGN APPROACH AND DETAILS**	9
	4.1 SYSTEM ARCHITECTURE	9
5.	**METHODOLOGY AND TESTING**	15
	5.1 Module Description	15
	5.2 Testing	22
6.	**PROJECT DEMONSTRATION – Implementations**	29
7.	**RESULT AND DISCUSSION**	32
8.	**CONCLUSION AND FUTURE ENHANCEMENTS**	37
9.	**REFERENCES**	40
	APPENDIX A – SAMPLE CODE	41

Note: *In accordance with the official course template guidelines, Section 3 is omitted in this report structure. The numbering therefore proceeds directly from Section 2 to Section 4, as in the template.*

1. INTRODUCTION

1.1 BACKGROUND

Machine-learning inference is steadily moving away from centralised cloud servers and onto resource-constrained edge devices such as microcontrollers, mobile processors and embedded accelerators. Zhou et al. [1] survey this shift under the name edge intelligence and argue that running inference close to the data source reduces latency, limits bandwidth consumption and improves privacy relative to cloud-only deployment. Chen and Ran [2] review how deep-learning workloads are adapted to operate within the memory, compute and energy limits of such devices.

These limits bear directly on numerical computation. Memory is scarce, arithmetic units may lack fast floating-point hardware, and on battery-powered platforms every additional instruction and every additional byte of memory traffic has a cost. A program written on the assumption of abundant 32-bit arithmetic can therefore be functionally correct yet impractical on such hardware. Reducing numerical precision is one of the most direct remedies: a signed 8-bit integer (INT8) occupies one byte, a quarter of the storage of a 32-bit value, and integer arithmetic can be cheaper than floating-point arithmetic on suitable hardware. Jacob et al. [3] describe a quantization scheme that allows inference to be carried out with integer-only arithmetic, and Krishnamoorthi [4] surveys quantization techniques for convolutional networks, including the calibration choices that keep quantization error acceptable.

In that literature, however, quantization is applied to an already-trained model and its safety is established empirically, by measuring the change in accuracy over a representative dataset. This project poses the same question inside a compiler: given a program, can the compiler decide *before execution* which individual numerical values may be represented in INT8 without overflow or loss of meaning, and rewrite only those? To answer it, the project builds a complete teaching-scale compiler for a small C-like numerical language. The compiler lowers programs to Three-Address Code (TAC), builds a control-flow graph (CFG), converts it to Static Single Assignment (SSA) form [9] and runs an abstract-interpretation analysis [5], [6] over exact interval and integrality facts. A value is retyped to INT8 only when the analysis shows that it lies in [-128, 127] and is integral on every path; every other value keeps its original 32-bit representation.

Alongside the compiler, the project provides **RANGE//LAB**, an interactive web workbench that exposes every intermediate representation and every quantization decision visually. The compiler is the primary contribution of the work; the workbench is its demonstration and inspection layer.

1.2 MOTIVATIONS

The project is motivated by the following observations.

- **Indiscriminate precision reduction is unsafe.** Replacing 32-bit values by INT8 can silently introduce overflow when a value leaves [-128, 127], incorrect representation when a value is not an integer, loss of exactness when intermediate results are confined to 8 bits, and corrupted downstream computation, because one unsafe conversion propagates to every consumer of the value.
- **Statistical validation gives no per-value statement.** Calibration-based quantization [3], [4] selects scales on a representative dataset and judges the outcome by end-to-end

accuracy. This is effective on average, but it says nothing about a specific value on every execution path.

- **Compile-time reasoning adds no run-time cost.** A decision taken statically costs nothing on the device, whereas a run-time range check would add instructions and branches on exactly the hardware that can least afford them.
- **A compiler already has the right structure.** Explicit temporaries (TAC), control-flow joins (CFG), single definitions (SSA) and data-flow propagation are the standard ingredients for tracking how a numerical property flows through a program. Abstract interpretation [5], [6] and widening [7] supply the theory, and Astrée [8] shows that interval reasoning can be applied to real embedded software.
- **Decisions can be explained.** Because source positions are preserved through every phase, each decision is reported with a reason, for example "possible overflow: 130 > 127" (Section 7.2), which makes the optimisation auditable.
- **Pedagogical value.** The project connects lexing, parsing, intermediate code generation, CFG construction, SSA, data-flow analysis and optimisation in one coherent application, and the workbench makes each stage visible during a demonstration.

Soundness is a design goal of the analysis: an INT8 decision is taken only when the inferred facts justify it, and any uncertainty is resolved in favour of the original representation. This report evaluates that behaviour empirically on a fixed set of benchmarks (Section 7). It does not offer a universal formal proof of the correctness of the implementation.

1.3 SCOPE OF THE PROJECT

The project is a teaching-scale compiler. It compiles a focused, C-like imperative language, called `.qc`, that was designed to expose compiler algorithms rather than to replace a production language. Table 1.1 states precisely what lies inside and outside the scope of the work.

Table 1.1: Scope of the project: implemented features and explicit non-goals

Implemented and verified (in scope)	Not implemented (out of scope)
Mini numerical language `.qc`: `int` and `float` variables, assignment, `if/else`, `while`, `print`, arithmetic and relational operators (IR types also include `bool` and `int8`)	Compilation of arbitrary C or C++ programs; import of PyTorch or ONNX computational graphs
Ten-stage compiler: lexer, parser, semantic analysis, TAC, CFG, SSA, range analysis, INT8 safety and quantization, diagnostics, simulator	Arrays, pointers, structures, heap allocation and tensors; boolean variables and logical operators (`&&`, `||`, `!`)
SSA-based interval and integrality analysis over exact rationals, with branch refinement, widening and narrowing	Native or LLVM code generation (for example ARM Cortex-M or RISC-V assembly, object files)
INT8 safety verification with operand-dependency blocking and selective retyping	Deployment to physical edge hardware
Software simulator that executes original and quantized IR and compares their output	Measured speed-up, latency or energy figures
RANGE//LAB workbench (React 19 front end, FastAPI back end) with ten pipeline-stage views	Scale-factor and zero-point calibration of fractional values
141 automated tests and a 12-benchmark reproducible evaluation harness	Relational abstract domains (octagons, polyhedra)

Throughout this report a feature is described as **implemented** only if it exists in the repository and was exercised in the recorded verification runs. Ideas that have not been implemented are collected in Section 8 and are never described in the present tense. Any memory saving reported is a theoretical static representation-width calculation (Section 7.5), not a measured hardware result.

Organisation of the report. Section 2 reviews the literature and states the objectives and the problem. Section 4 presents the design of the compiler and of the workbench. Section 5 describes each module and the testing methodology. Section 6 walks through a demonstration of the workbench. Section 7 reports and discusses the results, Section 8 concludes and lists future enhancements, Section 9 lists the references, and Appendix A contains the sample programs. This report builds on the Review 2 report, which documented the design baseline and the first working compiler core; Section 7.6 compares the two milestones.

2. PROJECT DESCRIPTION AND GOALS

2.1 LITERATURE REVIEW

The survey moves from the practical motivation for the project (edge constraints and quantization) towards the theoretical and systems foundations on which the compiler is built (static analysis, SSA and compiler infrastructure). All twelve sources are listed in Section 9.

2.1.1 Edge AI and efficient computation

Zhou et al. [1] review the architectures, frameworks and enabling technologies for running deep-learning models at the network edge and identify constrained memory, compute and energy as recurring obstacles that separate edge deployment from cloud deployment. Chen and Ran [2] give a complementary review of how deep-learning workloads are partitioned, compressed and adapted, covering model compression, hardware acceleration and edge–cloud collaboration. Both surveys point to the same requirement: a technique must reduce the resource cost of computation without needing specialised training infrastructure on the device. Reducing numerical precision is one of the few techniques that lowers memory footprint directly, without changing the structure of the program.

2.1.2 Quantization and reduced-precision computation

Jacob et al. [3] describe a quantization scheme, together with a co-designed training procedure, that allows inference to run with integer-only arithmetic on mobile CPUs; representing weights and activations in 8 bits instead of 32 reduces their storage roughly fourfold. Krishnamoorthi [4] gives a broader overview of quantization for convolutional networks, covering post-training quantization and quantization-aware training, and recommends per-channel weight quantization combined with per-layer activation quantization as a practical default for INT8 deployment. Both works rely on scale factors (and in some schemes zero-points) and on calibration, and both judge safety statistically, through the accuracy of the resulting model. This project adopts the same target representation, signed 8-bit integers in [-128, 127], but asks a different question: whether an individual value can be shown, before execution, never to leave that range and never to be fractional. Per-channel scaling, zero-point calibration and accuracy-based tuning are explicitly outside its scope.

2.1.3 Abstract interpretation, widening and range analysis

The theoretical basis of the range and integrality analysis is the abstract-interpretation framework of Cousot and Cousot. Their 1976 paper [5] introduces interval abstraction, in which the possible values of a variable at a program point are over-approximated by an interval [minimum, maximum]. Their 1977 paper [6] generalises this into a lattice-theoretic framework in which a static analysis is the construction or approximation of a fixed point of a monotone function. Interval lattices have infinite height, so naive fixed-point iteration over a loop need not terminate. Bourdoncle [7] formalises widening and narrowing operators and gives iteration strategies, based on a weak topological ordering of the control-flow graph, that guarantee termination while limiting the loss of precision. Blanchet et al. [8] describe Astrée, an interval- and abstract-interpretation-based analyser built to prove the absence of run-time errors, including arithmetic overflow, in large safety-critical embedded C programs, and report very few false alarms on the avionics software they studied. Astrée demonstrates that the kind of reasoning used here, bounding a variable's range in order to exclude a class of run-time error, is practical for embedded software.

2.1.4 SSA and compiler infrastructures

Cytron et al. [9] give the efficient dominance-frontier algorithm for inserting φ-functions and renaming variables so that every definition has a single, unambiguous static location. Because every SSA name has exactly one definition, a data-flow fact such as an interval and an integrality flag can be attached to the defining instruction, and a φ-node needs only to merge the facts of its incoming operands. Lattner and Adve [10] adopt SSA as the core of LLVM, a typed low-level representation on which analyses and transformations are applied uniformly across compile time, link time and run time. Ragan-Kelley et al. [11] separate the algorithm of an image-processing pipeline from its execution schedule in Halide, and Chen et al. [12] extend the idea to deep-learning workloads in TVM through graph-level and operator-level optimisations. These systems share a structural idea that this project borrows directly: numerical and hardware-aware optimisations are implemented as passes over an explicit intermediate representation, guided by static analysis, rather than as ad hoc source-level rewrites.

Table 2.1: Summary of the surveyed work and its relation to this project

Ref.	Work	Contribution	Relation to this project
[1]	Zhou et al., edge intelligence survey	Taxonomy of edge-AI architectures and constraints	Motivation (memory, compute, energy)
[2]	Chen and Ran, deep learning with edge computing	Review of compression, acceleration and offloading	Motivation for reduced precision
[3]	Jacob et al., integer-only inference	Integer-arithmetic-only quantization scheme	Source of the INT8 target representation
[4]	Krishnamoorthi, quantization whitepaper	Per-channel and per-layer quantization, calibration	Contrast: statistical, accuracy-based safety
[5]	Cousot and Cousot, 1976	Interval abstraction of variable values	Foundation of the range analysis
[6]	Cousot and Cousot, 1977	Lattice model and fixed-point approximation	Fixed-point framework for the analysis
[7]	Bourdoncle, 1993	Widening, narrowing and iteration strategies	Loop termination and precision recovery

Ref.	Work	Contribution	Relation to this project
[8]	Blanchet et al., Astrée	Industrial interval-based run-time-error analysis	Evidence that the approach is practical
[9]	Cytron et al., 1991	Dominance-frontier SSA construction	Algorithm used to build the SSA form
[10]	Lattner and Adve, LLVM	SSA-based typed IR and pass infrastructure	Design precedent for IR-based passes
[11]	Ragan-Kelley et al., Halide	Algorithm and schedule separation	Precedent for hardware-aware compiler passes
[12]	Chen et al., TVM	Optimising compiler stack for deep learning	Precedent; this project isolates one sub-problem

2.1.5 Synthesis

Read together, the literature traces a path to the proposed project. Edge-intelligence research establishes that memory, compute and energy constraints make reduced precision attractive. The quantization literature shows that INT8 is a practical representation while also showing that quantization is normally validated empirically for a whole model. Abstract interpretation supplies interval abstraction, fixed-point computation and widening, the machinery needed to decide safety for an individual value, and Astrée shows that this machinery works on real embedded code. SSA provides the representation in which such an analysis is naturally expressed, and LLVM, Halide and TVM confirm that treating numerical optimisation as an explicit compiler pass is a sound architectural choice.

2.2 RESEARCH GAP

None of the surveyed systems targets exactly this problem, but it would be inaccurate to claim that no existing work touches on any part of it. Quantization systems reason about numerical precision; Astrée and the abstract-interpretation literature reason about static value ranges; and LLVM, Halide and TVM show that compiler infrastructures can perform hardware- and precision-aware transformations. The gap identified here is one of scope and transparency rather than novelty:

- industrial quantization pipelines operate on trained models and validate safety statistically over a dataset [3], [4];
- industrial static analysers are large, highly tuned systems built for hundreds of thousands of lines of safety-critical code [8];
- general-purpose infrastructures expose numerical optimisations as a few passes inside a much larger system [10]–[12].

This project instead provides a compact pipeline in which SSA construction, range and integrality analysis, conservative safety decision-making and precision transformation are separate, individually inspectable stages over a small language, with the goal of making the underlying principles easy to study, trace and defend in a Compiler Design course. The Review 2 baseline left three precision and safety gaps open, and Review 3 closes them:

- **Branch conditions did not refine intervals.** The Review 2 analysis was flow-insensitive with respect to branch conditions. Review 3 adds path-sensitive refinement along true and false edges and detects statically unreachable paths.

- **Loops lost their bounds.** After widening, a loop counter in Review 2 became unbounded and was rejected. Review 3 adds a narrowing pass so that bounded counters recover tight intervals.
- **Intermediate overflow was not tracked.** Review 2 decided each value from its own interval. Review 3 adds an operand-dependency check in the quantizer: a value that is within [-128, 127] on its own is still left unchanged if any operand it consumes is not itself INT8-safe, for example when an upstream intermediate overflowed.

A fourth gap, the absence of any means of inspecting the compiler's internal state interactively, is addressed by the RANGE//LAB workbench.

2.3 OBJECTIVES

The project is guided by the following objectives. Table 2.2 records, for each one, the component that realises it and the evidence presented in this report.

1. Specify a small C-like numerical language (`.qc`) with a formal grammar, sufficient to express computations whose INT8 safety is worth analysing.
2. Implement a lexer, a parser and a semantic analyser that carry line and column positions through to later phases.
3. Lower validated programs to Three-Address Code, partition them into basic blocks and build a control-flow graph that handles conditional branches and loop back-edges.
4. Construct minimal SSA form using dominator trees, dominance frontiers and variable renaming.
5. Implement interval and integrality abstract interpretation over exact rational arithmetic, with monotone transfer functions for each supported operator.
6. Refine intervals along conditional edges (path-sensitive analysis) and detect statically unreachable paths.
7. Ensure termination on loops with widening and a sweep limit, and recover precision with narrowing.
8. Define conservative INT8 safety rules, block quantization when an upstream intermediate may have overflowed, and rewrite only approved instructions.
9. Provide a software simulator that executes the original and the quantized IR and compares their output.
10. Evaluate the system reproducibly with an automated test suite and a benchmark harness, reporting only measured or mathematically derived quantities.
11. Provide an interactive workbench that visualises each compiler phase and each quantization decision.

Table 2.2: Objectives, realising components and evidence

No.	Realised by	Principal evidence in this report	Status
1	Language definition and grammar	Section 4.1.4 (EBNF), Appendix A	Implemented
2	`src/lexer/`, `src/parser/`, `src/ast/`, `src/semantic/`	Sections 5.1.1 and 5.1.2	Implemented
3	`src/ir/`, `src/cfg/`	Sections 4.1.5 and 4.1.6	Implemented
4	`src/ssa/`	Section 4.1.7	Implemented
5	`src/analysis/abstract_value.py`, `transfer.py`	Sections 5.1.3 and 5.1.4	Implemented
6	`src/analysis/refinement.py`	Sections 5.1.5 and 7.3	Implemented (Review 3)
7	`src/analysis/range_analysis.py`	Section 5.1.6	Implemented (Review 3)
8	`src/quantization/safety.py`, `quantizer.py`	Sections 5.1.7 and 5.1.8	Implemented (Review 3)
9	`src/backend/simulator.py`	Sections 5.1.9 and 5.2.4	Implemented
10	`evaluate.py`, `tests/`	Sections 5.2 and 7	Implemented (Review 3)
11	`src/api/`, `web/`	Sections 4.1.3 and 6	Implemented (Review 3)

2.4 PROBLEM STATEMENT

> **Problem.** Given a program *P* written in the `.qc` language and converted to SSA form, decide statically, for every SSA value *v* of *P*, whether *v* may be represented as a signed 8-bit integer without changing the observable behaviour of the program, and rewrite exactly those values for which the decision is positive. Every value for which safety cannot be established must keep its original 32-bit representation.

The analysis assigns to each SSA value *v* an abstract value *a(v)* = (interval, integral) that over-approximates the set of values *v* can take on any execution path, together with a diagnostic division-by-zero flag. With the signed INT8 domain defined as

$$\mathrm{INT8} = \{\ \mathrm{n} \in \mathbb{Z} : -128 \leq \mathrm{n} \leq 127\ \} \tag{2.1}$$

a value is judged SAFE when its interval is bounded and contained in [-128, 127], it is guaranteed integral, it is defined by a supported operation, and no division-by-zero risk is flagged. A SAFE value is actually rewritten only if every operand it consumes is itself rewritten, and every constant operand fits INT8 (Sections 5.1.7 and 5.1.8). The problem therefore has three parts: soundly inferring abstract values, deciding eligibility, and transforming and validating the program.

The solution is bound by four design constraints that recur throughout the report:

- **Conservatism:** whenever the analysis cannot establish safety, the value keeps its original representation.
- **Separation of concerns:** the analysis only labels values; the transformation pass only acts on values already labelled safe.

- **Traceability:** each decision can be traced back to a source position and a stated reason.
- **No run-time overhead:** the decision is made entirely at compile time.

The project does not attempt to prove the correctness of the implementation for all programs, to reproduce the statistical calibration of neural-network quantization, or to measure the performance of quantized code on hardware.

4. DESIGN APPROACH AND DETAILS

4.1 SYSTEM ARCHITECTURE

The system has two cooperating parts: a Python compiler and quantization engine, and the RANGE//LAB web application that presents the output of every stage. This section describes the principles that guided the design, the compiler pipeline, the intermediate representations and the full-stack architecture.

4.1.1 Design principles

- **Conservative safety.** The analysis over-approximates the values a variable can take, and the safety rule resolves every uncertainty (unknown range, possible overflow, possible fraction, possible division by zero) in favour of the original representation.
- **Decide, then transform.** The safety pass only labels values; the quantizer only rewrites values that have been labelled safe and whose operands are also rewritten. Each part can be tested independently.
- **SSA as the vehicle for sparse analysis.** SSA gives every value exactly one definition, so the analysis reduces to a per-instruction transfer function plus a merge rule at φ-nodes [9]. It makes the analysis clean; it is not an optimisation in itself.
- **Source traceability.** Line and column positions are attached to tokens, AST nodes and SSA instructions, so a decision can be reported against its originating source location.
- **Decoupled presentation.** The workbench consumes serialised compiler results over a thin API; it does not re-implement any analysis.

4.1.2 Compiler pipeline

A source program flows through ten stages, each consuming the representation produced by the previous one (Figure 4.1). Table 4.1 lists the module that implements each stage, its input and output, and its responsibility. The driver `src/pipeline.py` orchestrates the stages, and `src/main.py` exposes them on the command line through the options `--emit`, `--narrow`, `--widen-after`, `--prune-unreachable` and `--inputs`. The workbench presents the same ten stages as its stage views.

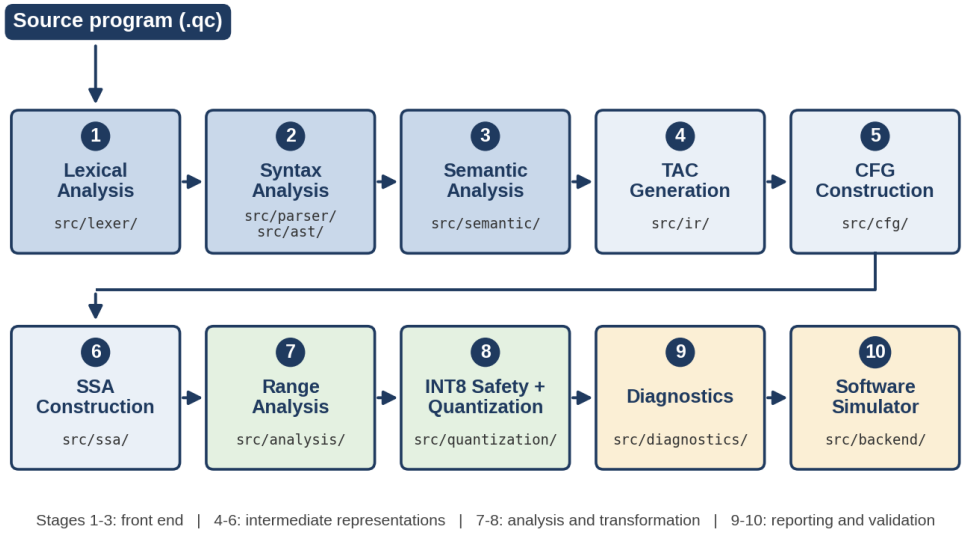


Figure 4.1: Overall pipeline of the Range-Aware Edge AI Quantization Compiler: ten stages and the source packages that implement them.

Architecture diagram prepared for this report from the repository structure.

Table 4.1: Compiler stages, modules and responsibilities

#	Stage	Module	Input → Output	Responsibility
1	Lexical analysis	`src/lexer/tokens.py`, `lexer.py`	Source text → tokens	Hand-written scanner with line and column tracking; skips comments; exact int and float literals
2	Syntax analysis	`src/parser/parser.py`, `src/ast/`	Tokens → AST	Recursive-descent parser with table-driven, left-associative binary operator levels
3	Semantic analysis	`src/semantic/analyzer.py`	AST → checked AST	Declaration and use checks, duplicate declarations, type and assignability checks
4	TAC generation	`src/ir/lowering.py`, `tac.py`, `values.py`	Checked AST → TAC	Lowering with explicit temporaries and labels
5	CFG construction	`src/cfg/cfg.py`, `dominators.py`	TAC → CFG	Leaders, basic blocks, edges; dominator tree and dominance frontiers
6	SSA construction	`src/ssa/construction.py`, `verify.py`	CFG → SSA program	Minimal φ placement and renaming; structural verification
7	Range analysis	`src/analysis/`	SSA → range environment	Interval and integrality analysis, branch refinement, widening, narrowing
8	INT8 safety and quantization	`src/quantization/safety.py`, `quantizer.py`	Ranges → verdicts → quantized SSA	Per-value safety decision; operand-dependency blocking; selective retyping
9	Diagnostics	`src/diagnostics/report.py`	Results → report	One record per SSA value with source location, range and reason
10	Simulator	`src/backend/simulator.py`	Two IRs → outputs	Executes the original and the quantized IR and compares the printed output

4.1.3 Full-stack architecture

The workbench is a decoupled client–server application (Figure 4.2). The browser client, a single-page application written in React and TypeScript and built with Vite, contains the Monaco code editor with a custom `.qc` tokenizer, a stage navigator with ten views and status badges, a React Flow visualiser for the CFG, an SSA variable inspector drawer and a diff view that compares pre- and post-quantization IR. It communicates over HTTP and JSON with a FastAPI server (`src/api/server.py`, port 8000 in the documented development setup). The server executes the compiler in-process and converts the Python AST, CFG, SSA, range, verdict and simulator objects to JSON through a dedicated serialiser (`src/api/serializer.py`). Four endpoints are exposed:

- `POST /api/compile` runs the full pipeline on the submitted source and returns the result of every stage;
- `POST /api/simulate` executes the original and quantized IR on demand;
- `GET /api/examples` returns the built-in teaching examples shown in the example selector;
- `GET /api/health` is the health check behind the “Compiler Engine Online” indicator.

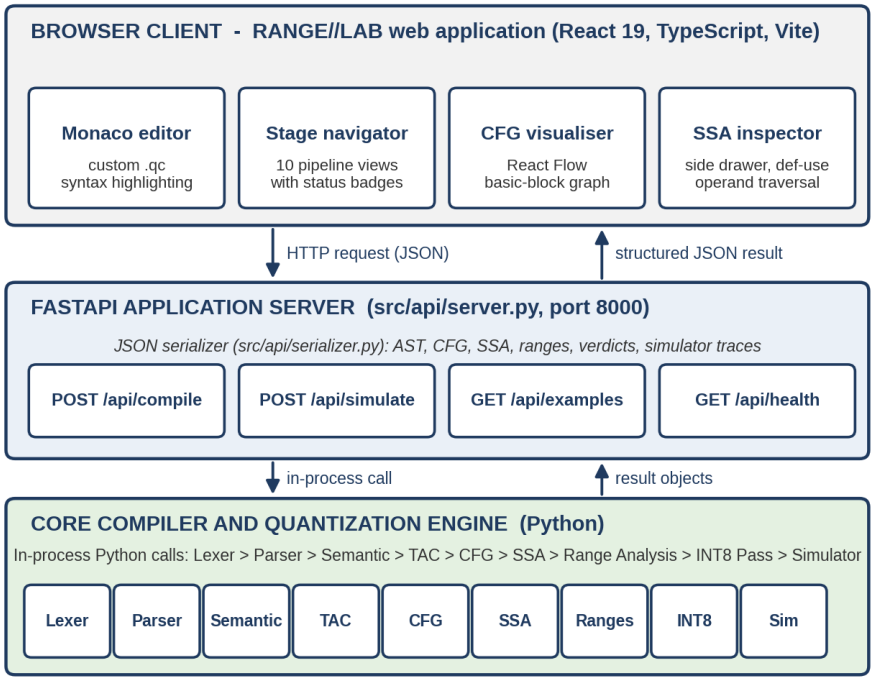


Figure 4.2: Full-stack architecture of RANGE//LAB: browser client, FastAPI server with JSON serialiser, and the core compiler and quantization engine.
Architecture diagram prepared for this report from the repository structure.

4.1.4 Source language and type system

The `.qc` language is a small C-like imperative language. A program is a sequence of declarations, assignments, `if/else` and `while` statements, bare blocks and `print` statements. Variables are declared with the types `int` and `float`; the IR additionally uses `bool` for comparison results, `int8` for quantized values and names the floating-point type `fp32`. A declaration without an initialiser denotes an externally supplied input, whose value is unknown to the analysis. Line comments (`//`) and block comments (`/* */`) are skipped by the lexer. The

grammar, in EBNF, is given below; binary operators are left-associative, and the levels run from equality (lowest) to multiplication and division (highest).

```
program      ::= statement*
statement    ::= declaration | assignment | if | while | print | block
declaration ::= ("int" | "float") IDENT ("=" expr)? ";"
assignment  ::= IDENT "=" expr ";"
if          ::= "if" "(" expr ")" statement ("else" statement)?
while       ::= "while" "(" expr ")" statement
print       ::= "print" expr ";"
block       ::= "{" statement* "}"

expr        ::= equality
equality    ::= relational (("==" | "!=") relational)*
relational  ::= additive (("<" | "<=" | ">" | ">=") additive)*
additive    ::= multiplicative (("+" | "-") multiplicative)*
multiplicative ::= unary (("*" | "/") unary)*
unary       ::= "-" unary | primary
primary     ::= IDENT | INT_LITERAL | FLOAT_LITERAL | "(" expr ")"
```

4.1.5 Three-Address Code generation

After semantic analysis, the AST is lowered to Three-Address Code in which every instruction applies at most one operator to at most two operands. Intermediate results are named explicitly (`t1`, `t2`, …), which is the granularity at which range analysis operates: an expression such as `(a + b) * c` cannot be usefully analysed as a single unit, but its decomposition into temporaries can. Control flow is expressed with labels and jumps. For the program `if_else_phi.qc` (Appendix A, Program 4) the compiler emits the following 13 TAC instructions, labels included, which agrees with the count shown in the workbench:

```
x = 10
y = 20
max = 0
t1 = x > y
if t1 goto L1
goto L2
L1:
max = x
goto L3
L2:
max = y
L3:
print max
```

4.1.6 Basic blocks and the control-flow graph

Basic blocks are formed with the standard leader-based algorithm: the first instruction, the target of any jump and the instruction that follows any jump or conditional jump are leaders, and each block extends from a leader up to, but excluding, the next leader. Edges are added for fall-through execution, unconditional jumps and the two outcomes of a conditional branch; a `while` loop contributes a back-edge from the end of the body to the loop-condition block. The CFG is the structure over which SSA construction and range analysis run; without it there is no well-defined join point at which differently valued definitions of a variable must be merged. Figure 4.3 shows the graph that the workbench draws for `if_else_phi.qc`.

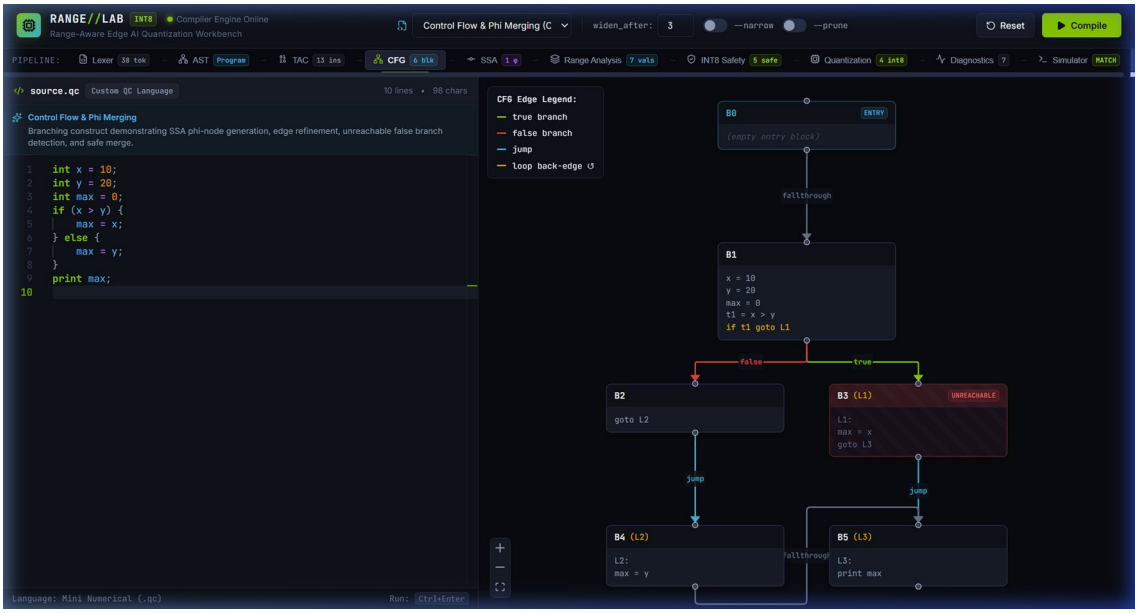


Figure 4.3: Interactive control-flow graph of if_else_phi.qc (six basic blocks, B0 to B5), rendered with React Flow. Edges are labelled fall-through, jump, true and false; B3 is marked UNREACHABLE.
Evidence file: 06-control-flow-graph.png

The graph has six basic blocks. B0 is the empty entry block. B1 holds the first four assignments and the conditional jump `if t1 goto L1`. The false edge leads to B2 (`goto L2`) and then, by a jump, to B4 (`L2: max = y`); the true edge leads to B3 (`L1: max = x`, `goto L3`); B4 falls through to B5 (`L3: print max`), which is also the target of the jump from B3, and is therefore the join block. A legend colour-codes the edge kinds. Only B3 carries an UNREACHABLE badge: with x = 10 and y = 20 the condition x > y is statically false, so the true edge is never taken. The join block B5 is reachable through B2 and B4 and is not marked. An earlier version of the graph view also tagged B5 as unreachable. The reachability computation of the API serialiser (`compute_reachable_cfg_blocks` in `src/api/serializer.py`) now follows only branch-feasible edges, and the regression test `test_cfg_reachability_if_else_phi` in `tests/api/test_api.py` covers this program.

4.1.7 SSA construction

SSA construction follows the algorithm of Cytron et al. [9] in four steps: compute the dominator tree of the CFG (`src/cfg/dominators.py`); compute the dominance frontier of every block; insert a φ-node for a variable at every block in the iterated dominance frontier of the blocks that define it, which gives minimal SSA form; and rename every definition and use with a fresh version number (x becomes x.1, x.2, and so on). A verification module (`src/ssa/verify.py`) accompanies the construction. The workbench SSA stage offers three sub-views: the SSA program, the table of φ-nodes and the dominance tree.

For `if_else_phi.qc` the definition `max = 0` becomes `max.1`, the assignment on the false path becomes `max.2` and the assignment on the true path becomes `max.3`. The two versions meet at the join block, where exactly one φ-node is placed (the SSA badge reads “1 φ”). Figure 4.4 shows the φ-node view.

```
max.4 = phi(B4: max.2, B3: max.3)
```

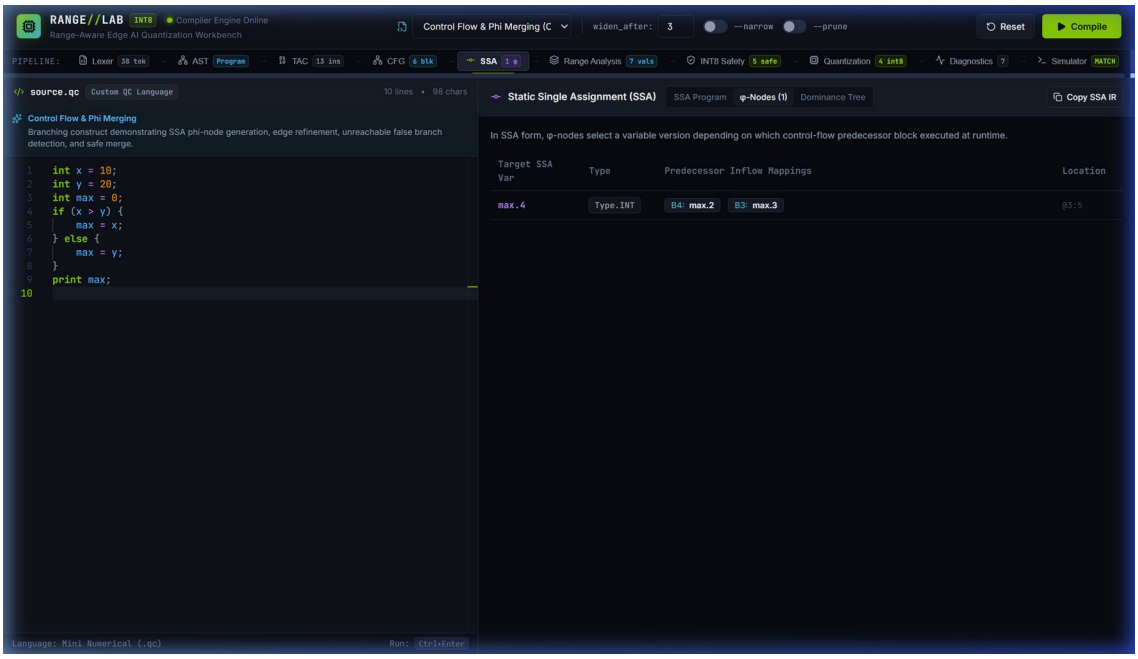


Figure 4.4: SSA φ-node view for if_else_phi.qc: target max.4 (Type.INT) with predecessor inflow mappings B4: max.2 and B3: max.3, at source location 3:5.
Evidence file: 10-tac-ssa-phi.png

Because every SSA name has one definition, the range analysis can be described as a transfer function per instruction together with a merge at φ-nodes. The same program in the Review 2 report produced the φ-node `max.4 = phi(B3: max.3, B4: max.2)`, so the SSA numbering is unchanged between the two milestones.

4.1.8 Source-level traceability

Line and column positions are recorded for every token and copied into AST nodes (shown as @line:col in Section 5.1.1) and into SSA instructions. A decision can therefore be reported against a precise source location, as the inspector of Section 6.3 shows for the variable `z.1`. The diagnostics module (`src/diagnostics/report.py`) emits one record per SSA value in the form used in Review 2, for example for the overflowing sum of Section 7.2:

```
Line 3, Column 5
Variable: z.1
Inferred range: [130, 130]
Integral: YES
INT8 Safety: REJECTED
Reason: possible overflow: 130 > 127
```

5. METHODOLOGY AND TESTING

5.1 Module Description

This section describes, in pipeline order, what each module does and the algorithms it uses. Modules marked as new in Review 3 were absent from, or substantially extended after, the Review 2 baseline.

5.1.1 Lexical analysis and parsing

The lexer (`src/lexer/lexer.py`) is a hand-written character scanner that converts source text into typed tokens; every token carries the line and column at which it begins, and numeric literals are stored as exact fractions. The parser (`src/parser/parser.py`) is a hand-written recursive-descent parser that implements the grammar of Section 4.1.4, resolving the binary-operator levels with a table-driven loop, and produces the node classes defined in `src/ast/nodes.py`. Figure 5.1 shows the result for the bounded-loop program.

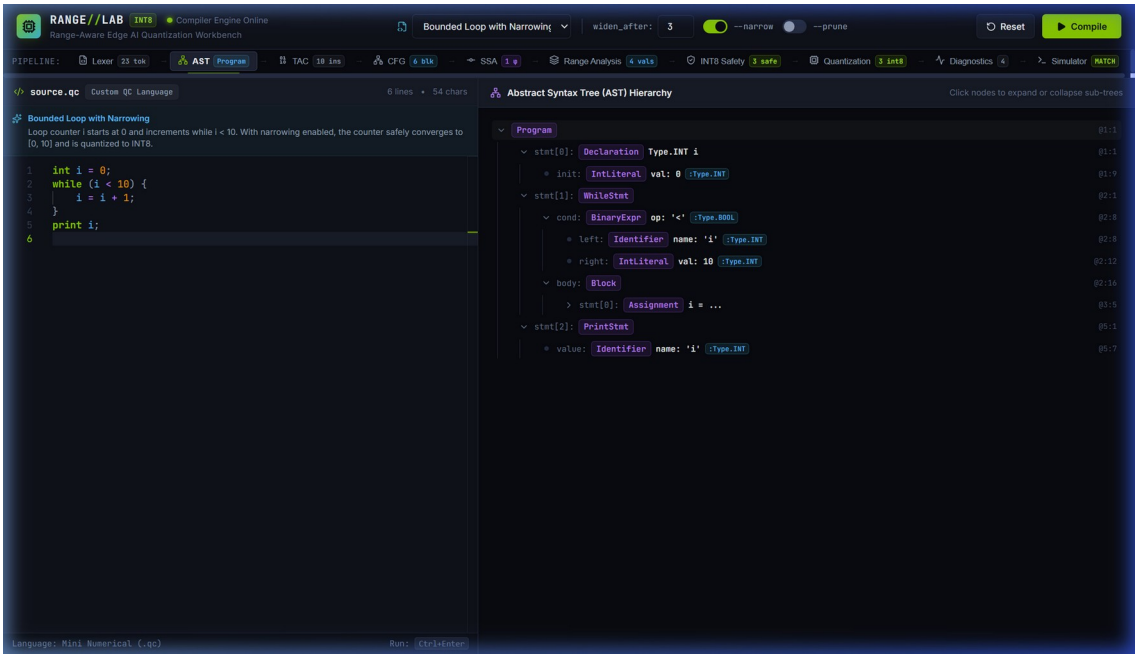


Figure 5.1: Frontend view for the bounded-loop program (int i = 0; while (i < 10) { i = i + 1; } print i;): token count in the pipeline bar and the collapsible AST with node types, inferred types and @line:col coordinates.

Evidence file: 09-lexer-ast.png

The pipeline bar reports 23 tokens, which corresponds to the 22 lexemes of the source plus an end-of-input token. The AST has the root `Program` with three statements: a `Declaration` of `Type.INT i` at 1:1 whose initialiser is an `IntLiteral` 0 at 1:9; a `WhileStmt` at 2:1 whose condition is a `BinaryExpr` with operator < (typed `Type.BOOL`) over the identifier `i` at 2:8 and the literal 10 at 2:12, with a `Block` body at 2:16 containing an `Assignment` at 3:5; and a `PrintStmt` at 5:1.

5.1.2 Semantic analysis

The semantic analyser (`src/semantic/analyzer.py`) walks the AST with a scoped symbol table. It reports the use of undeclared variables, assignment to undeclared variables and duplicate declarations, infers expression types, and checks that assignments and conditions are type-correct, including the rejection of an implicit narrowing of a float into an integer variable.

Each expression node of the result carries its type, as in the `:Type.INT` and `:Type.BOOL` annotations of Figure 5.1. Only semantically valid programs are lowered to TAC.

5.1.3 Abstract value domain

The range analysis describes each SSA variable by an abstract value (`src/analysis/abstract_value.py`):

$$D = I \times B \tag{5.1}$$

$$I = \{\ [l, u] : l, u \in \mathbb{Q} \cup \{-\infty, +\infty\}, l \leq u\ \} \cup \{\ \bot\ \} \tag{5.2}$$

- **Interval (I).** Bounds are exact rational numbers (Python `Fraction`), so the analysis introduces no floating-point rounding error: a bound such as 13.5 is represented exactly. The element $\bot$ marks a value on a statically unreachable path. An interval with an infinite end is treated as UNKNOWN by the safety rule.
- **Integrality (B).** The flag `integral` is True only if every run-time value on every path is guaranteed to be an integer. Float-typed constants and float-typed variables are never integral.
- **Diagnostic payload.** A division-by-zero flag and a textual reason travel with the value for reporting; they do not take part in the comparison that decides convergence.

At a φ-node the abstract values of the incoming operands are joined: the result interval is the smallest interval containing the intervals of the reachable operands, and the integrality flag is the conjunction of their flags. A value that is only sometimes integral is therefore non-integral, which is what a safety decision that must hold on every path requires. An unreachable operand ($\bot$) does not affect the result, and an UNKNOWN operand makes the result UNKNOWN.

5.1.4 Interval transfer functions

Each supported operator has a monotone transfer function over intervals (`src/analysis/transfer.py`), listed in Table 5.1 for X = [a, b] and Y = [c, d]. The formulas follow the interval-analysis tradition of Cousot and Cousot [5], [6].

Table 5.1: Abstract transfer functions

Operator	Result interval	Integrality of the result
Addition	[a + c, b + d]	X.integral ∧ Y.integral
Subtraction	[a − d, b − c]	X.integral ∧ Y.integral
Multiplication	[min(ac, ad, bc, bd), max(ac, ad, bc, bd)]	X.integral ∧ Y.integral
Division	If 0 ∈ [c, d]: UNKNOWN with the division-by-zero flag set. Otherwise the interval spanned by the four quotients a/c, a/d, b/c, b/d	Always False (conservative)
Unary negation	[−b, −a]	X.integral
Relational (<, ≤, >, ≥, ==, !=)	[1, 1] if the relation holds for every pair of points, [0, 0] if it holds for none, otherwise [0, 1]; the result is typed bool	True

Multiplication considers all four endpoint products because the signs of the operands decide which combination gives the extremes. Division is conservatively non-integral, and a divisor interval that contains zero has no sound finite quotient interval.

5.1.5 Path-sensitive branch refinement (new in Review 3)

When a basic block ends in a conditional jump, the analysis refines the intervals of the compared operands separately along the true and the false edge (`src/analysis/refinement.py`). Operands may be variables or constants; a comparison between two variables refines both. Where the compared values are integral, strict bounds are shifted by one because the values are discrete. Table 5.2 lists the constraint that is intersected with the current interval of a variable x compared with a constant c; the false edge uses the negated condition.

Table 5.2: Branch refinement constraints for a variable x and a constant c

Condition	True edge	False edge
x < c	x ∈ [−∞, c − 1]	x ∈ [c, +∞]
x ≤ c	x ∈ [−∞, c]	x ∈ [c + 1, +∞]
x > c	x ∈ [c + 1, +∞]	x ∈ [−∞, c]
x ≥ c	x ∈ [c, +∞]	x ∈ [−∞, c − 1]
x == c	x ∈ [c, c]	x ≠ c (trims an interval end only)
x != c	x ≠ c (trims an interval end only)	x ∈ [c, c]

If the intersection of the current interval with a constraint is empty, the environment on that edge is marked ⊥ (unreachable) and the values defined on that path are identified as dead. Float operands are refined with unshifted bounds, and the test suite covers equality and inequality at a boundary. The effect on a concrete program is analysed in Section 7.3. The optional `--prune-unreachable` mode, which skips infeasible edges instead of marking them unreachable, is examined in Section 5.2.8.

5.1.6 Fixed-point iteration, widening and narrowing (new in Review 3)

The analysis (`src/analysis/range_analysis.py`) visits the CFG in reverse post-order and repeats sweeps until the environment stops changing. Because the interval lattice has infinite height, a loop-header φ-node can keep changing indefinitely; following Bourdoncle [7], widening is applied to a loop-header φ-node that is still changing after N sweeps (default N = 3, set with `--widen-after`). In the default mode the widened value becomes UNKNOWN. With the option `--narrow` the widening is bound-wise, sending only the bound that keeps moving to infinity:

$$[l_1, u_1] \nabla [l_2, u_2] = [\ (l_2 < l_1\ ?\ -\infty : l_1),\ (u_2 > u_1\ ?\ +\infty : u_1)\ ] \tag{5.3}$$

When the iteration has converged in the narrowing mode, up to five descending passes recompute each widened φ-node from the facts of its incoming edges, which now include the branch-refined loop condition, and intersect the result with the current value; the other instructions are then recomputed:

$$x' = x \sqcap F(x) \tag{5.4}$$

A sweep limit guards against non-termination: if the analysis has not converged within the limit, every value falls back to UNKNOWN. For the program `int i = 0; while (i < 10) { i = i + 1; }` the two modes give different results. Without `--narrow` the loop-header value i.2 is UNKNOWN, i.3 has an unbounded lower end, only i.1 is quantized, and the analysis converges in five sweeps (variable counts 4 / 1 / 1 / 0 / 3 in the notation of Section 5.2.3). With

`narrow`, shown in Figures 5.2 and 5.3, i.1 = [0, 0], the φ-node i.2 = phi(B1: i.1, B4: i.3) = [0, 10] (badge “widened”) and i.3 = i.2 + 1 = [1, 10]; all three are integral, lie inside [-128, 127] and are rewritten to `int8`, and the analysis converges in six sweeps (counts 4 / 3 / 3 / 0 / 1). The relational temporary `t1 = i.2 < 10` is rejected because the quantization pass does not rewrite comparisons.

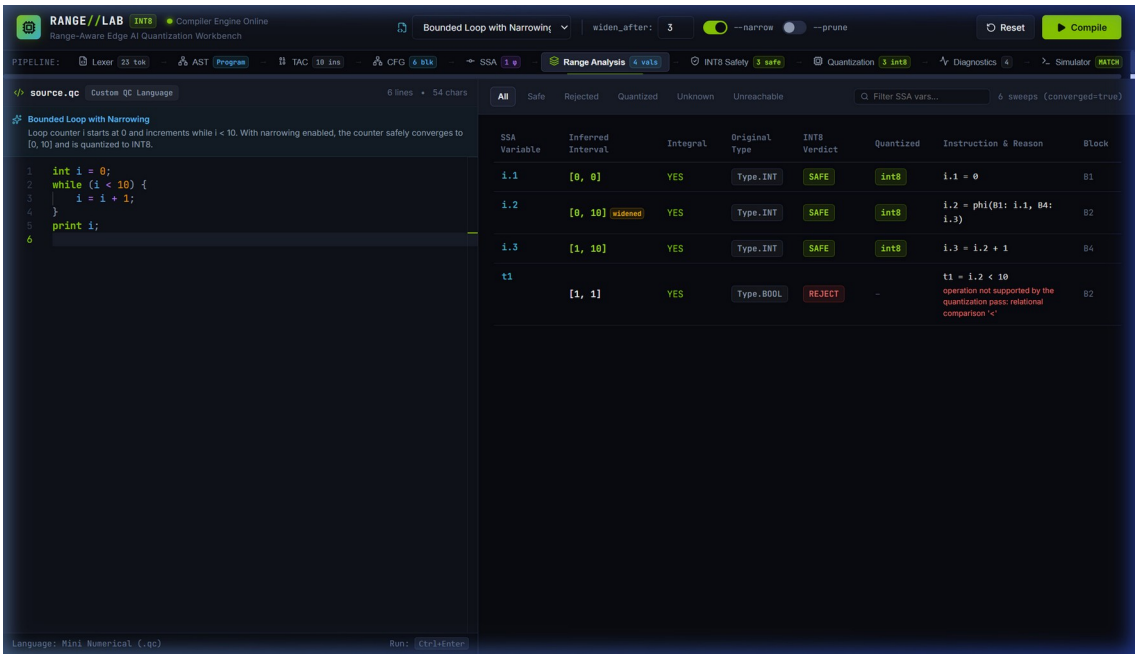


Figure 5.2: Range analysis table for the bounded-loop program with narrowing enabled: inferred interval, integrality, original type, INT8 verdict, quantized type, defining instruction with reason and block, with filter pills, a search box and the convergence indicator (6 sweeps, converged).

Evidence file: 11-range-analysis.png

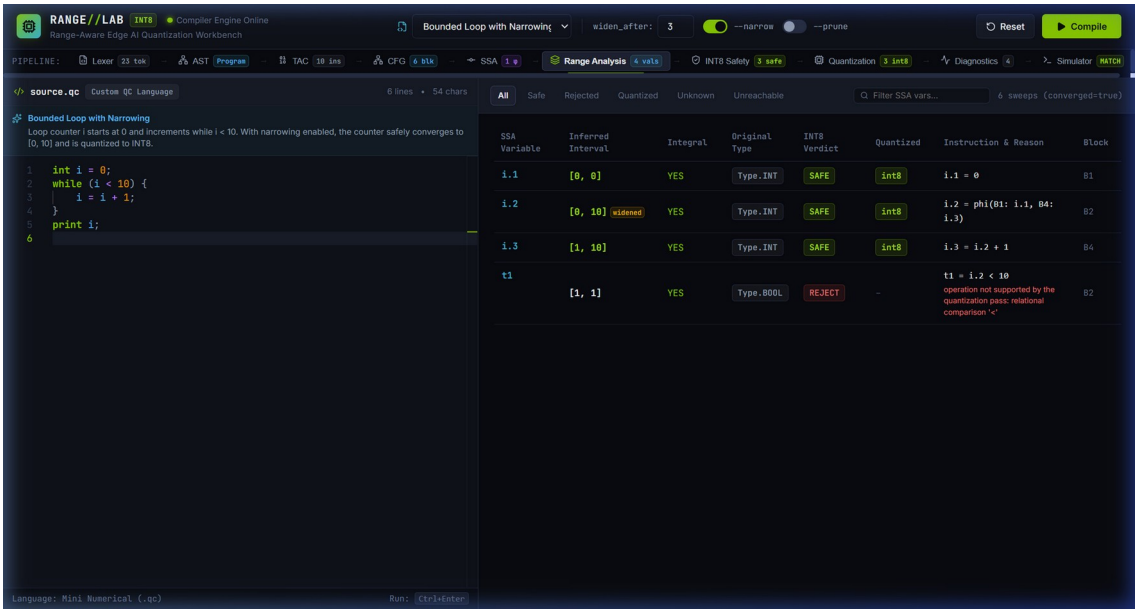


Figure 5.3: A second capture of the same bounded-loop analysis: the loop-header φ-node i.2 carries [0, 10] (flagged widened) and i.3 carries [1, 10], so the loop variables qualify for INT8.

Evidence file: 08-loop-narrowing.png

Figure 5.2 shows the table and its controls: the filter pills All, Safe, Rejected, Quantized, Unknown and Unreachable, a search field and the convergence counter. Both captures show the final, narrowed state; the widening-only state described above is a property of the algorithm and

was checked by running the compiler without `--narrow`, and is not shown in the figures. Narrowing needs a bounding exit condition: a loop whose counter has none stays UNKNOWN and is kept in its original form. The narrowed table lists the comparison temporary t1 as [1, 1]; this reflects the refined in-loop state and is not a bound on the exit test, which is false when i reaches 10. Because comparison results are never rewritten, it affects no quantization decision.

5.1.7 INT8 safety rules

The safety pass (`src/quantization/safety.py`) takes the range environment and assigns a verdict to every SSA value. A value is judged SAFE if and only if all six per-value conditions of Table 5.3 hold. A value on a statically unreachable path is rejected with the reason “unreachable or infeasible control-flow path”.

Table 5.3: Per-value INT8 safety conditions

#	Condition	Meaning
1	Known interval	Both bounds of the interval are finite; UNKNOWN and half-bounded intervals are rejected.
2	Lower bound	The minimum is at least -128.
3	Upper bound	The maximum is at most 127. The boundaries -128 and 127 themselves are safe; -129 and 128 are rejected.
4	Integrality	The value is integral on every path. Float values and division results are rejected.
5	Supported operation	The defining operation is a copy, a φ, addition, subtraction, multiplication or unary negation. Division, comparisons and externally supplied inputs are not supported.
6	No division-by-zero risk	No division-by-zero flag is attached to the value.

Figure 5.4 shows the safety view for the straight-line arithmetic program.

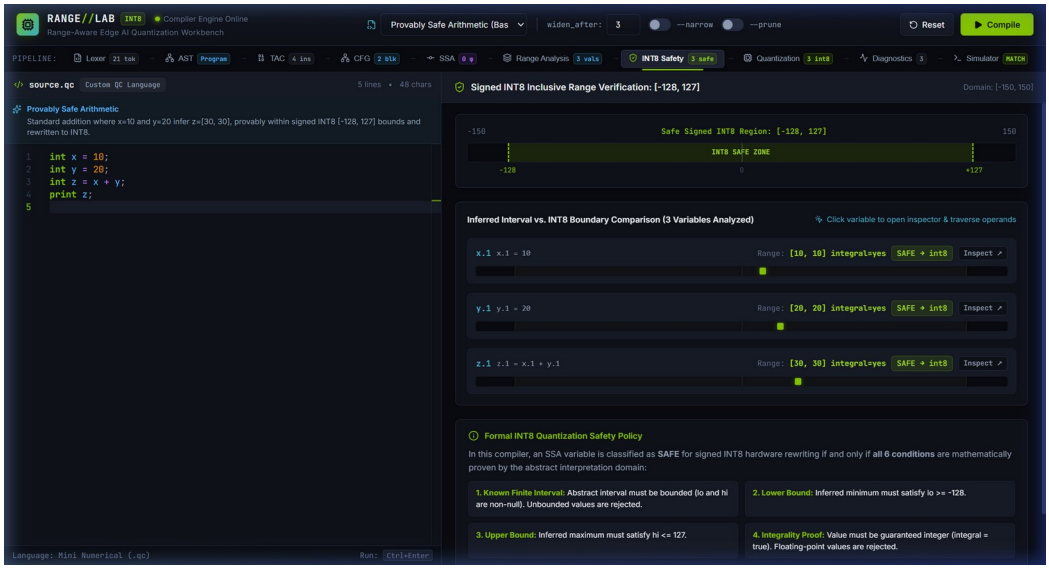


Figure 5.4: INT8 safety view for safe_arithmetic.qc (x = 10, y = 20, z = x + y): the inferred intervals [10, 10], [20, 20] and [30, 30] are placed against the signed INT8 region [-128, 127] drawn on a [-150, 150] scale, and each variable is labelled SAFE → int8.

Evidence file: 02-int8-safety.png

Each variable row shows its interval, its integrality, a verdict badge and a bar that positions the interval inside the INT8 region, and offers an Inspect control that opens the SSA inspector (Section 6.3). The panel “Formal INT8 Quantization Safety Policy” at the bottom of the view states that a value is SAFE if and only if all six conditions hold; the first four visible in the capture are a known finite interval, a lower bound of at least -128, an upper bound of at most 127 and a proof of integrality.

Passing the safety conditions does not by itself rewrite a value. The quantizer (Section 5.1.8) additionally requires that every operand a value consumes is itself rewritten and that every constant operand fits in INT8. The workbench therefore shows four verdicts. **SAFE** means the value passes Table 5.3 and is rewritten (shown as “SAFE → int8”). **BLOCKED** means it passes Table 5.3 but keeps its original type because an operand is not rewritten. **REJECT** means a condition fails, for example overflow, non-integrality or an unsupported operation. **UNREACHABLE** marks a value on a path proven infeasible. In the counts of Section 7, “Safe” includes blocked values and “INT8” counts only the values actually rewritten.

5.1.8 Selective quantization and operand-dependency blocking

The quantizer (`src/quantization/quantizer.py`) starts from the set of SAFE values and repeatedly removes a value from the set if any operand it consumes is not in the set, or if a constant operand does not fit in INT8, until no more values are removed. The values that remain are retyped from `int` to `int8` (with their constant operands); every other instruction, including the operands of an unsafe expression, keeps its original type. A φ-node counts as consuming all of its incoming operands, so a φ-node is blocked when any incoming operand is not rewritten, including an operand defined on a statically unreachable path (Section 7.3).

The rule prevents an expression from being rewritten after an upstream intermediate has overflowed. The program `intermediate_overflow` declares `a = 100`, `b = 50`, `c = a + b` and `d = c - 50`. Here c = 150 exceeds 127 and is rejected, while d evaluates to [100, 100], which is inside the INT8 range; because the operand c.1 is not rewritten, d.1 is blocked. The compiler emits:

```
a.1 : int8 = 100
b.1 : int8 = 50
c.1 : int = a.1 + b.1
d.1 : int = c.1 - 50
print d.1
```

Figure 5.5 shows the side-by-side comparison in the workbench, whose header reports two variables rewritten and one blocked. The blocking operates along def-use chains; the pass does not apply algebraic re-association to find an equivalent expression that avoids the overflow.

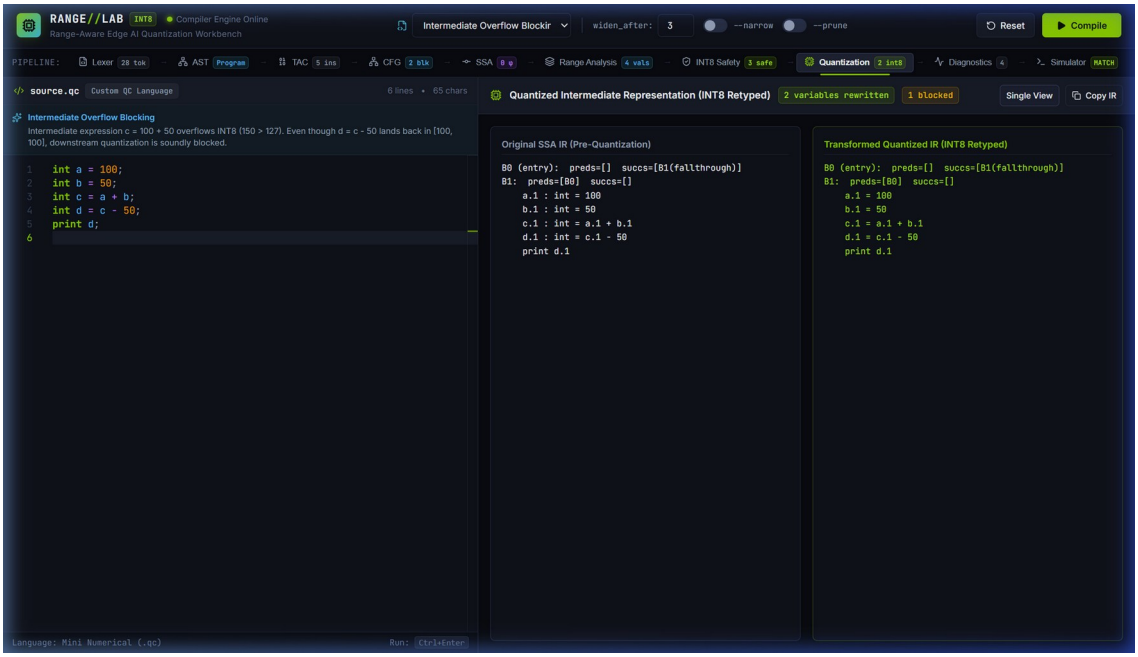


Figure 5.5: Side-by-side comparison of the original SSA IR and the transformed IR for the intermediate-overflow program (a = 100, b = 50, c = a + b, d = c - 50). The header reports 2 variables rewritten and 1 blocked.

Evidence file: 13-quantized-ir-diagnostics.png

5.1.9 Software simulator

The simulator (`src/backend/simulator.py`) executes the original SSA program and the quantized SSA program and compares their printed output. The original program is executed with Python's arbitrary-precision integers and floating-point numbers; no 32-bit wrap-around is emulated, and the 32-bit width of the baseline is an assumption used only for the storage estimate of Section 7.5. In the quantized program, values and constants typed `int8` are reduced to the signed 8-bit range by two's-complement wrap-around, not by saturation. Division by zero, missing inputs and runaway loops are reported as errors (a step limit stops infinite loops). The two runs are called equivalent when their output lists are equal (Section 5.2.4). The simulator is an in-memory validation tool; it is not an LLVM back end, a native code emitter or a hardware driver, and it does not measure performance.

5.1.10 Summary of module status

Table 5.4: Implementation status of the project modules

#	Module	Source files	Status	Section
1	Lexical analyser	`src/lexer/`	Completed	5.1.1
2	Syntax parser and AST	`src/parser/`, `src/ast/`	Completed	5.1.1
3	Semantic analyser	`src/semantic/`	Completed	5.1.2
4	TAC lowering	`src/ir/`	Completed	4.1.5
5	CFG builder and dominators	`src/cfg/`	Completed	4.1.6
6	SSA builder	`src/ssa/`	Completed	4.1.7
7	Abstract value domain	`src/analysis/abstract_value.py`	Completed	5.1.3
8	Transfer functions	`src/analysis/transfer.py`	Completed	5.1.4

#	Module	Source files	Status	Section
9	Branch refinement	`src/analysis/refinement.py`	Completed (Review 3)	5.1.5
10	Range analysis engine	`src/analysis/range_analysis.py`	Completed (Review 3)	5.1.6
11	INT8 safety pass	`src/quantization/safety.py`	Completed	5.1.7
12	Quantization rewriter	`src/quantization/quantizer.py`	Completed (Review 3)	5.1.8
13	Diagnostics reporter	`src/diagnostics/`	Completed	4.1.8
14	Software simulator	`src/backend/simulator.py`	Completed	5.1.9
15	Driver and command line	`src/pipeline.py`, `src/main.py`	Completed	4.1.2
16	FastAPI server and serialiser	`src/api/`	Completed (Review 3)	4.1.3
17	RANGE//LAB front end	`web/`	Completed (Review 3)	6

5.2 Testing

5.2.1 Strategy and environment

Verification has four layers: automated unit and integration tests for every stage, a reproducible benchmark evaluation harness, a build and lint check of the web front end, and live inspection of the workbench, which the screenshots in this report document. The recorded runs used a Windows x64 machine with Python 3.10 or later (the console path in Figure 5.6 indicates a Python 3.13 installation). The source code is kept in the project repository (https://github.com/yoswinner/Range-Aware-Edge-AI-Quantization-Compiler). The results of this section were reproduced from commit `05853b0`; the three regression tests, the one analysis correction and the two interface labels of Section 5.2.8 were made in the working copy after that commit, and the test counts below include them.

5.2.2 Unit and integration test suite

The test suite uses Python's standard `unittest` framework. Discovery across the `tests/` directory finds 17 test modules containing **141 tests**; all 141 passed, with no failures and no errors (result OK). Table 5.5 breaks the tests down by module. At commit `05853b0` the suite contained 138 tests, including `test_cfg_reachability_if_else_phi` for the CFG reachability labelling of Section 4.1.6; the three further tests are the pruning regression tests of Section 5.2.8. By comparison, the Review 2 baseline had 20 tests, which ran in 0.018 seconds.

Table 5.5: Unit and integration tests by module (17 modules, 141 tests)

Test module	Area covered	Tests
`tests/range_analysis/ test_range_analysis.py`	Abstract domain and transfer functions	27
`tests/analysis/test_branch_refinement.py`	Path-sensitive refinement and pruning mode	18
`tests/backend/test_simulator.py`	Software simulator	11
`tests/quantization/test_quantization.py`	Quantization pass	10

Test module	Area covered	Tests
`tests/integration/test_pipeline.py`	End-to-end pipeline	10
`tests/quantization/test_safety_edge_cases.py`	INT8 boundary and safety edge cases	9
`tests/api/test_api.py`	FastAPI server and CFG reachability	9
`tests/analysis/test_analysis.py`	Analysis, loops and widening	8
`tests/parser/test_parser.py`	Parser	7
`tests/ssa/test_ssa.py`	SSA construction	7
`tests/analysis/test_loop_narrowing.py`	Loop narrowing	6
`tests/cfg/test_cfg.py`	CFG construction	4
`tests/ir/test_tac.py`	TAC lowering	4
`tests/lexer/test_lexer.py`	Lexer	4
`tests/semantic/test_semantic.py`	Semantic analysis	4
`tests/integration/test_boolean_comparisons.py`	Boolean comparisons	2
`tests/ast_tests/test_ast.py`	AST nodes	1
Total	**17 modules**	**141**

Table 5.6 relates selected claims made in this report to the tests that verify them.

Table 5.6: Selected claims and the tests that verify them

Claim	Verifying tests
The INT8 boundary is inclusive	`test_exact_signed_int8_lower_boundary_minus_128`, `test_exact_signed_int8_upper_boundary_127`; `test_boundary_overflow_minus_129`, `test_boundary_overflow_128` (all in `test_safety_edge_cases.py`)
Intermediate overflow blocks downstream quantization	`test_intermediate_overflow_blocks_quantization` (`test_quantization.py`); `test_intermediate_overflow_blocks_downstream_quantization` (`test_safety_edge_cases.py`)
Non-integral values are rejected	`test_non_integral_float_rejected`, `test_division_result_rejected_as_non_integral` (`test_quantization.py`)
Division by zero is detected statically	`test_division_by_zero_rejected` (`test_quantization.py`); `test_division_by_zero_transfer` (`test_analysis.py`)
Branch refinement detects unreachable paths	`test_impossible_branch_returns_unreachable` (`test_branch_refinement.py`); `test_unreachable_value_rejected` (`test_safety_edge_cases.py`)
Loop narrowing recovers bounded intervals	`test_increasing_counter_bounded_with_narrowing`, `test_unconstrained_expanding_loop_remains_unknown` (`test_loop_narrowing.py`)
φ-nodes are placed using dominance frontiers	`test_ssa_dominance_frontiers_computed`, `test_ssa_phi_insertion` (`test_ssa.py`)
Original and quantized programs print the same output	`test_simulate_programs_equivalence_on_safe_program` (`test_simulator.py`)

The simulator's two's-complement wrap-around is implemented in `src/backend/simulator.py` but is not exercised by a dedicated test; on the benchmarks it is never triggered, because the safety pass admits only values inside [-128, 127].

Figure 5.6, which follows the description of the evaluation harness in Section 5.2.3, presents the verification summary in two panes; the note below states what the figure is.

> **Nature of Figure 5.6.** This figure is a verification summary rendered programmatically from the recorded output of the two commands of Section 5.2.6 and laid out like a terminal window. It is not an unedited terminal screenshot. Pane (a) is abridged: the figure itself states that 95 passing tests are omitted for the scroll view, and it ends with "Ran 138 tests in 0.316s, OK"; it records the run at commit `05853b0`, so the final total of 141 tests (Table 5.5) is not shown in it. Pane (b) shows the full 12-row evaluation report. Run times of a sub-second test suite vary between runs, and the figure should be read as a summary of results, not as a timing measurement. The commands can be re-run from the repository to reproduce the output.

5.2.3 Benchmark evaluation harness

The harness `evaluate.py` (with `tests/evaluation.py`) compiles and analyses 12 curated benchmark programs, whose sources are embedded in `tests/evaluation.py`, and prints

one row per benchmark and a summary. For each benchmark it reports the number of analysed SSA variables (SSA), the number labelled safe (Safe), the number actually rewritten to INT8 (INT8), the number blocked by an operand that is not rewritten (Block), the number rejected (Reject), the result of the simulation comparison (Sim Eq) and the status of the safety check (Safety, with a violation count). Only one benchmark, `bounded_loop_counter`, is run with narrowing enabled; all benchmarks use the default widening threshold and leave `--prune-unreachable` off. Results are analysed in Section 7.



Figure 5.6(a) Unit-test run

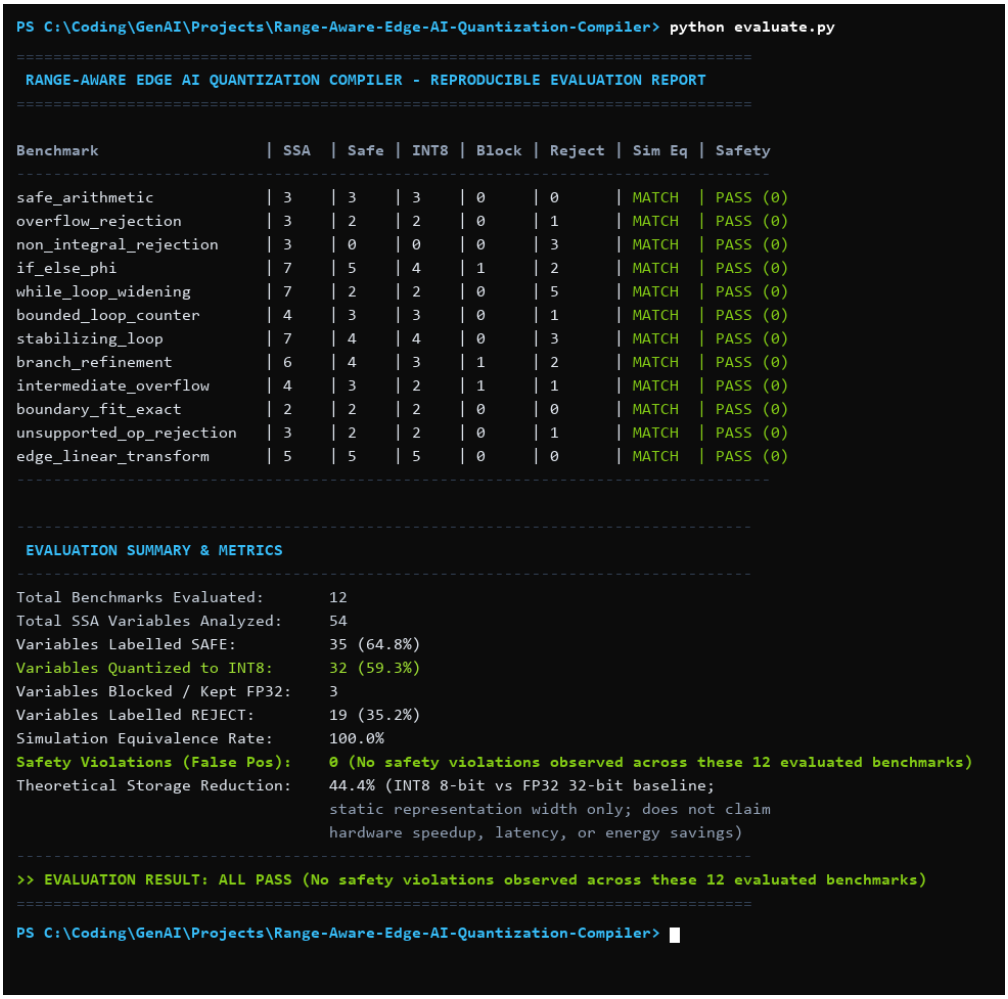


Figure 5.6(b) Evaluation report

Figure 5.6: Programmatically rendered verification summary (not an authentic terminal screenshot): (a) abridged unittest output ending “Ran 138 tests … OK” (the run at commit 05853b0, before the three pruning regression tests of Section 5.2.8); (b) the evaluate.py report.

Evidence file: 14-verification-results.png (shown here as two cropped panes)

5.2.4 Simulation equivalence validation

For each benchmark the simulator runs the original and the quantized program and compares their output. Figure 5.7 shows the comparison for `safe_arithmetic.qc`: both runs print 30 and the workbench reports the programs as equivalent. The interface notes that an empirical match is not a universal algebraic equivalence; in this report a match is empirical validation, for the executed inputs, that the quantized program reproduces the original output, and not a proof for all inputs.

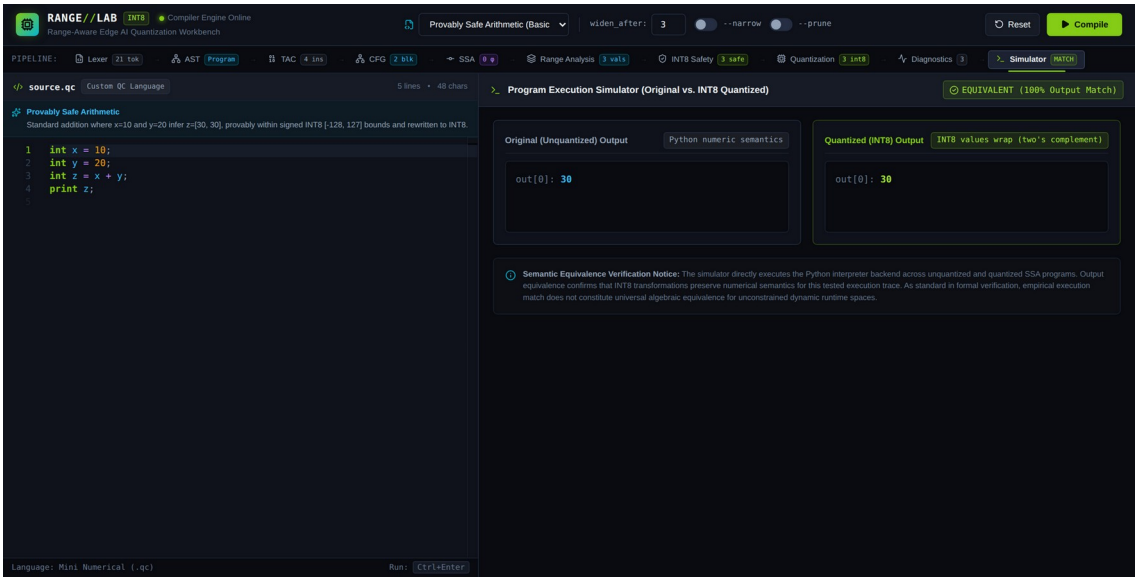


Figure 5.7: Dual-engine simulator for safe_arithmetic.qc: the original output (out[0]: 30, labelled “Python numeric semantics”) and the quantized output (out[0]: 30, labelled “INT8 values wrap (two’s complement)”) are compared, and the interface reports the programs as equivalent (100% output match).
Evidence file: 07-simulation-equivalence.png (recaptured from the running workbench after the label correction)

The two pane labels of Figure 5.7 describe the implemented semantics (Section 5.1.9): the original program is executed with Python numeric semantics, and values typed INT8 wrap by two’s-complement arithmetic. In earlier captures these panes were labelled “Baseline 32-bit” and “Signed 8-bit Clamped”, which did not match the simulator, because the original run is not limited to 32 bits and INT8 values are not saturated. The labels were corrected in `web/src/components/views/SimulatorView.tsx`, and Figure 5.7 was recaptured from the real workbench built from the corrected source. Because the safety pass admits only values that lie in [-128, 127], neither wrap-around nor saturation changes any quantized value on the executed benchmarks, so the reported outputs are unaffected.

5.2.5 Front-end and API verification

The production build of the web front end (`npm run build`, TypeScript project build followed by Vite) completes without errors, including after the label correction of Section 5.2.8; it prints an advisory that the main script chunk exceeds 500 kB. The lint check (`npm run lint`) reports 0 errors and 62 warnings. The API layer is covered by 9 of the 141 tests. The interface behaviour itself was inspected live in the browser against the running FastAPI server, and the screenshots of Sections 4 to 7 document the inspected states.

5.2.6 Reproducing the results

The reported numbers can be regenerated from the repository root with the commands below.

```
# full unit and integration test suite
python -m unittest discover -s tests -p "test_*.py" -v

# reproducible evaluation over the 12 benchmarks
python evaluate.py

# compile one program, show ranges and quantized IR, run the simulator
python -m src.main examples/safe_arithmetic.qc --emit ranges quantized simulate

# production build of the web front end
cd web && npm run build
```

5.2.7 Limits of the testing evidence

- The benchmark set is small and hand-written for a teaching-scale language; it demonstrates behaviour on these 12 programs, not on programs in general.
- Simulation equivalence and the absence of observed safety violations are empirical results on the executed inputs and benchmarks.
- No run-time, latency, energy or hardware-power measurement was performed; the project contains no hardware back end.
- Timing of the test suite varies between runs and is reported only to show that the suite is fast.

5.2.8 Final verification pass

A final pass compared this report with the repository at commit `05853b0` and with the running workbench. It led to the following changes and findings.

- **Optional pruning mode (--prune-unreachable).** An exploratory run of `branch_refine.qc` with the option enabled gave a wider merge interval (y.4 = [10, 20] instead of [10, 10]) and a reachable interval for the dead branch (y.2 = [20, 20]). The cause was that the infeasible edge was skipped, but the block behind it was then analysed with the global facts instead of being treated as dead. The analysis now records dead blocks explicitly: a block with no feasible incoming edge is dead, the values it defines are unreachable, dead predecessors are ignored at joins and φ-nodes, and dead blocks are skipped by narrowing (`src/analysis/range_analysis.py`). Three regression tests in `tests/analysis/test_branch_refinement.py` cover the corrected behaviour; they fail on the code of commit `05853b0` and pass with the correction. With the correction, the pruned and default modes give identical ranges and quantization decisions on all 12 benchmarks and on all 9 example files, each with and without narrowing, and every pruned run converges with equivalent simulation. The default mode used for all reported results was not changed, and the pruning option remains outside the 12-benchmark evaluation.

Dead branch followed by another conditional (default mode). The comparison also exposed a limitation of the default mode, which has no counterpart in the benchmarks. For the program below, the default analysis gives the loop-header value an interval narrower than the values it takes (i.2 = [0, 0] and t1 = [1, 1], although i reaches 200) and labels the live loop body UNREACHABLE; with `--prune-unreachable --narrow` the corrected analysis gives i.2 = [0, 200]. In the default run the affected values were rejected or blocked rather than quantized, and both simulator runs print 200.

```
int i = 0;
int s = 0;
while (i < 200) {
    if (i > 500) {
        s = s + 100;
    } else {
        s = s + 1;
    }
    i = i + 1;
}
print s;
```

- **Randomised differential test.** 1,250 randomly generated programs (750 on the corrected code and 500 on the code of commit `05853b0`), each compiled in four configurations (default and pruned, with and without narrowing), produced no mismatch between the original and the quantized output. Three further runs of 250 programs timed out and are excluded. This test did not reproduce an unsafe quantization, but it is not a proof, and the behaviour above should be treated as a known limitation (Section 8.3).
- **Simulator labels.** The two pane labels of the simulator view were corrected as described in Section 5.2.4; the front end was rebuilt and re-checked (Section 5.2.5).

6. PROJECT DEMONSTRATION – Implementations

This section demonstrates the RANGE//LAB workbench, the interactive presentation layer of the compiler. The workbench does not contain any analysis of its own; every value it displays is produced by the Python compiler engine and delivered through the FastAPI server (Section 4.1.3).

6.1 Workbench Overview

Figure 6.1 shows the workbench with the built-in example “Provably Safe Arithmetic” (`safe_arithmetic.qc`) compiled and the Range Analysis stage selected. The interface has three working areas:

- **Header and controls.** An example selector, the `widen_after` threshold field (set to 3), the `--narrow` and `--prune` toggles (both off here; the latter corresponds to the command-line option `--prune-unreachable`), a Reset button and a Compile button (also triggered by Ctrl+Enter). The indicator “Compiler Engine Online” reflects the health check of the server.
- **Pipeline bar.** Ten stage tabs, each with a live status badge. For this program the badges read: Lexer 21 tokens, AST Program, TAC 4 instructions, CFG 2 blocks, SSA 0 φ,

Range Analysis 3 values, INT8 Safety 3 safe, Quantization 3 int8, Diagnostics 3 and Simulator MATCH.

- **Editor and results panel.** The Monaco editor on the left shows the `.qc` source with line numbers and syntax highlighting. The panel on the right shows the selected stage; here the range table lists x.1 = [10, 10], y.1 = [20, 20] and z.1 = [30, 30] in block B1, each integral, SAFE and quantized to `int8`, and reports “2 sweeps (converged=true)”.

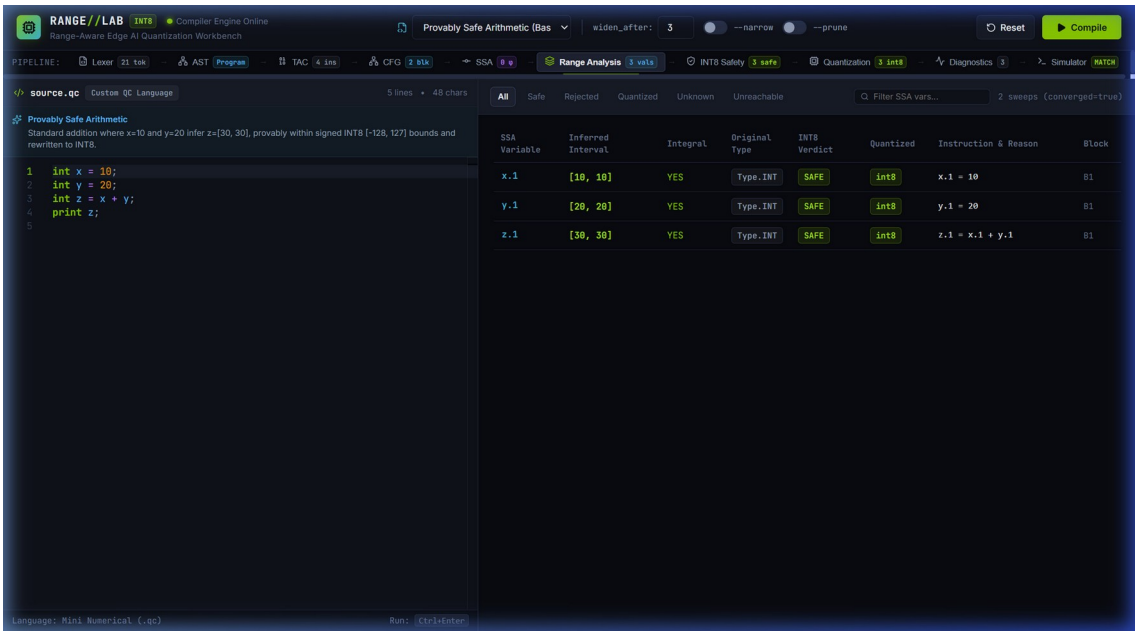


Figure 6.1: RANGE//LAB integrated compiler workbench for safe_arithmetic.qc: Monaco editor, control panel, ten-stage pipeline bar with status badges, and the range analysis table.
Evidence file: 01-main-workbench.png

Figure 6.1 demonstrates that the editor, the live Python compiler back end and the stage views are integrated end to end. It demonstrates the user-interface workflow and the system integration; it does not by itself establish the correctness of the analysis, which is evaluated in Sections 5 and 7.

6.2 Demonstration Walkthrough

A typical demonstration proceeds as follows.

1. Select an example from the selector (or type a program into the editor) and press Compile. The pipeline bar is populated with the status badge of every stage.
2. Walk through the stages from left to right: the token count and AST (Figure 5.1), the TAC listing, the CFG (Figure 4.3), the SSA program with its φ-nodes (Figure 4.4), the range table (Figure 5.2), the INT8 safety view (Figure 5.4), the quantized IR diff (Figure 5.5), the diagnostics and the simulator (Figure 5.7).
3. Toggle `--narrow` and change `widen_after` to observe how loop analysis precision changes.
4. In the INT8 safety view, press Inspect on a variable to open the SSA inspector drawer (Section 6.3) and follow operands through the def-use chain (Section 6.4).
5. Open the simulator stage to confirm that the original and quantized programs print the same output.

6.3 Inspecting an SSA Variable

Clicking the variable z.1 in the safety view opens the SSA variable inspector, a side drawer that gathers every fact the compiler holds about one SSA value (Figure 6.2). For z.1 the drawer shows:

- the defining instruction `z.1 = x.1 + y.1`, the operation (binary addition), the basic block B1 and the source location Line 3, Column 5;
- its two operands, x.1 with range [10, 10] and y.1 with range [20, 20], each integral, as clickable links;
- the abstract range and integrality: inferred interval [30, 30], integral, with division-by-zero risk “None (Safe)”;
- the INT8 safety verdict SAFE, with “Retyped to INT8: YES (int8)”;
- its downstream use, in block B1: `print z.1`.

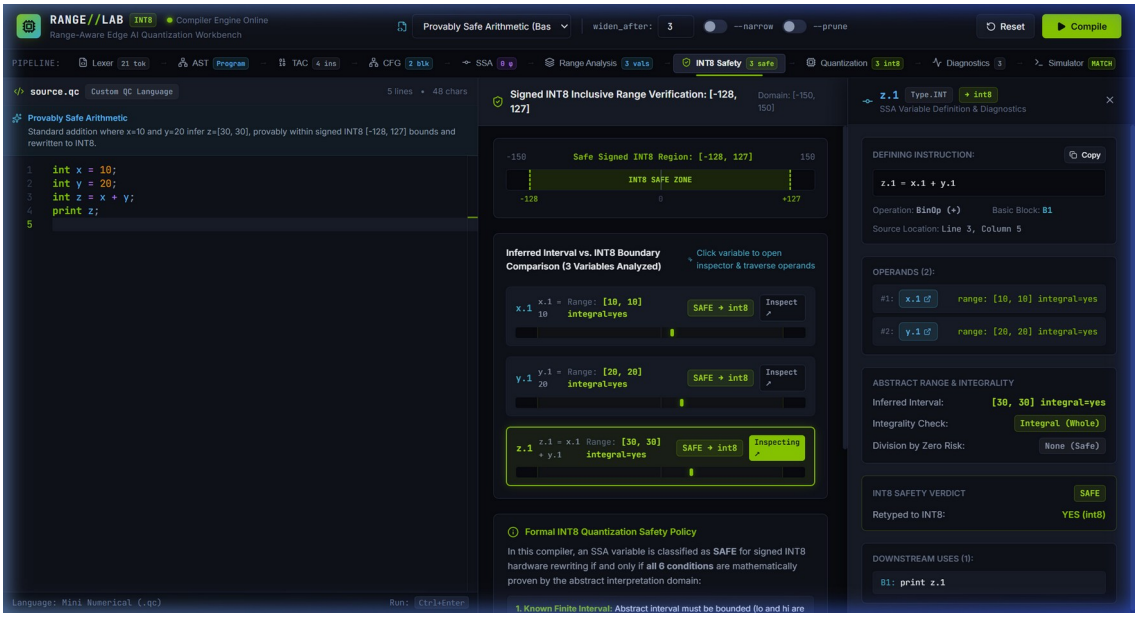


Figure 6.2: SSA variable inspector drawer for z.1 in safe_arithmetic.qc, showing the defining instruction z.1 = x.1 + y.1, block B1, source location 3:5, operands, interval [30, 30], integrality, verdict SAFE → int8 and downstream use print z.1.

Evidence file: 04-ssa-inspector-z1.png

6.4 Operand Traversal along the Def-Use Chain

Clicking the operand link x.1 inside the drawer updates the drawer in place, without leaving the safety view (Figure 6.3). The drawer now describes x.1: its defining instruction `x.1 = 10` (a copy of the literal 10, with the literal shown as its only operand), block B1, source location Line 1, Column 5, interval [10, 10], integral, verdict SAFE retyped to `int8`, and its downstream use `z.1 = x.1 + y.1` in block B1. The use entry links back to z.1, so the user can move both up (to operands) and down (to uses) along the def-use chain.

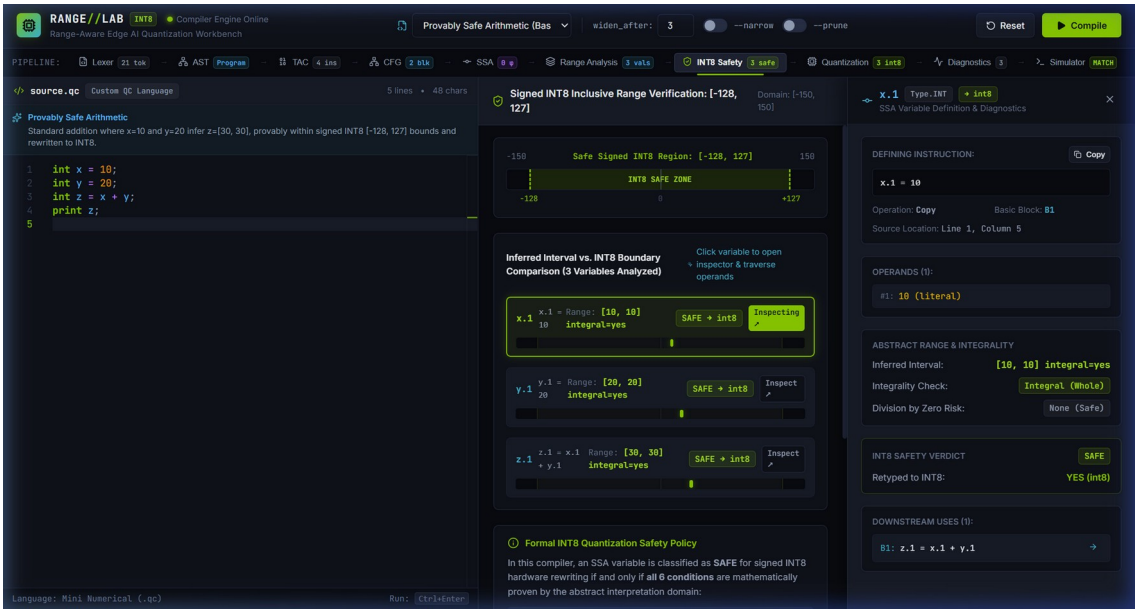


Figure 6.3: In-place operand navigation: after selecting the operand x.1, the drawer shows x.1 = 10 in block B1 with interval [10, 10], verdict SAFE → int8 and the downstream use z.1 = x.1 + y.1.
Evidence file: 05-ssa-inspector-x1.png

Figures 6.2 and 6.3 illustrate straight-line arithmetic; φ-nodes and loop-header values are examined in Figures 4.4 and 5.2.

6.5 Other Demonstrated Scenarios

The same workflow was used to demonstrate the behaviours analysed in Section 7: rejection of a possible overflow (Section 7.2), path-sensitive refinement with an unreachable branch (Section 7.3), loop narrowing (Figure 5.3) and operand-dependency blocking (Figure 5.5). The workbench is the inspection layer of the project; the compiler engine behind it is the contribution that Sections 4 and 5 describe.

7. RESULT AND DISCUSSION

7.1 Benchmark Evaluation Results

Running `python evaluate.py` on the 12 benchmark programs analysed 54 SSA variables in total. Table 7.1 reproduces the output of the harness (summarised in Figure 5.6) and Figure 7.1 plots its classification columns.

Table 7.1: Reproducible evaluation results for the 12 benchmarks

Benchmark	SSA	Safe	INT8	Block	Reject	Sim Eq	Safety
`safe_arithmetic`	3	3	3	0	0	MATCH	PASS (0)
`overflow_rejection`	3	2	2	0	1	MATCH	PASS (0)
`non_integral_rejection`	3	0	0	0	3	MATCH	PASS (0)
`if_else_phi`	7	5	4	1	2	MATCH	PASS (0)
`while_loop_widening`	7	2	2	0	5	MATCH	PASS (0)
`bounded_loop_counter`	4	3	3	0	1	MATCH	PASS (0)
`stabilizing_loop`	7	4	4	0	3	MATCH	PASS (0)
`branch_refinement`	6	4	3	1	2	MATCH	PASS (0)
`intermediate_overflow`	4	3	2	1	1	MATCH	PASS (0)
`boundary_fit_exact`	2	2	2	0	0	MATCH	PASS (0)
`unsupported_op_rejection`	3	2	2	0	1	MATCH	PASS (0)
`edge_linear_transform`	5	5	5	0	0	MATCH	PASS (0)
Total	**54**	**35 (64.8%)**	**32 (59.3%)**	**3 (5.6%)**	**19 (35.2%)**	**100.0% MATCH**	**0 observed**

Column meanings follow Section 5.2.3, and each benchmark is analysed with the default settings except `bounded_loop_counter`, which uses `--narrow`. In every row Safe = INT8 + Block and Safe + Reject = SSA, so that, over the whole suite, 35 = 32 + 3 and 35 + 19 = 54. Of the 54 variables, 32 (59.3%) were rewritten to INT8, 3 (5.6%) passed the safety conditions but were blocked because an operand is not rewritten, and 19 (35.2%) were rejected; the remaining 22 variables keep their original type.

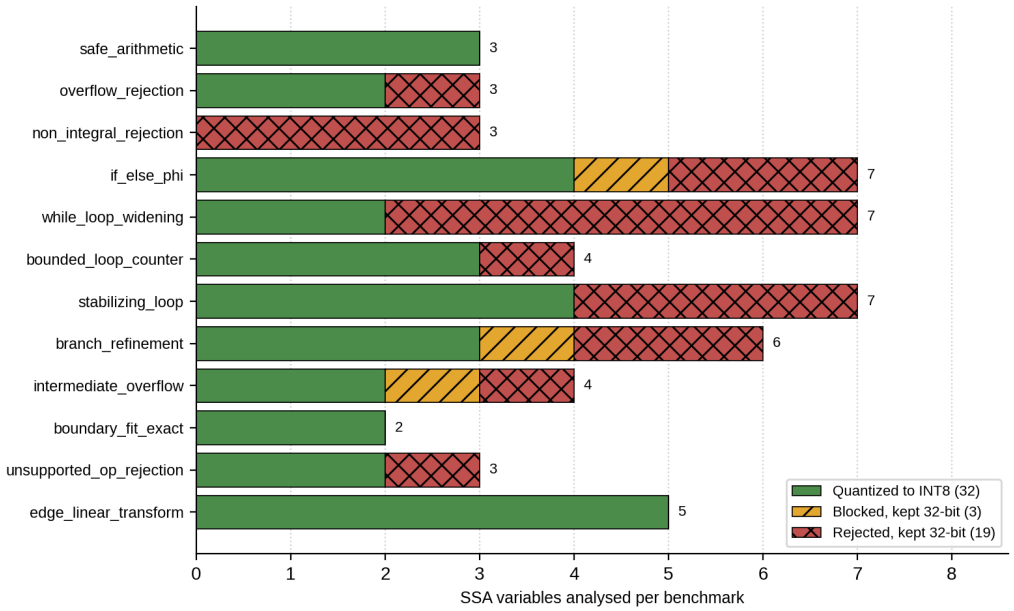


Figure 7.1: Classification of the SSA variables of each benchmark into quantized, blocked and rejected, plotted from Table 7.1.

Chart prepared for this report from the values in Table 7.1.

7.2 Safety Rejection Cases

Direct overflow. The program `overflow_rejection.qc` declares `x = 100` and `y = 30` and computes `z = x + y`. The analysis infers x.1 = [100, 100] and y.1 = [30, 30], both integral and inside [-128, 127], so both are rewritten to INT8. The sum z.1 = [130, 130] exceeds 127, and the safety pass rejects it with the reason “possible overflow: 130 > 127”; z.1 keeps its 32-bit int type (Figure 7.2). Rejection of values at or below -129 is exercised by the automated edge-case tests rather than by this figure.

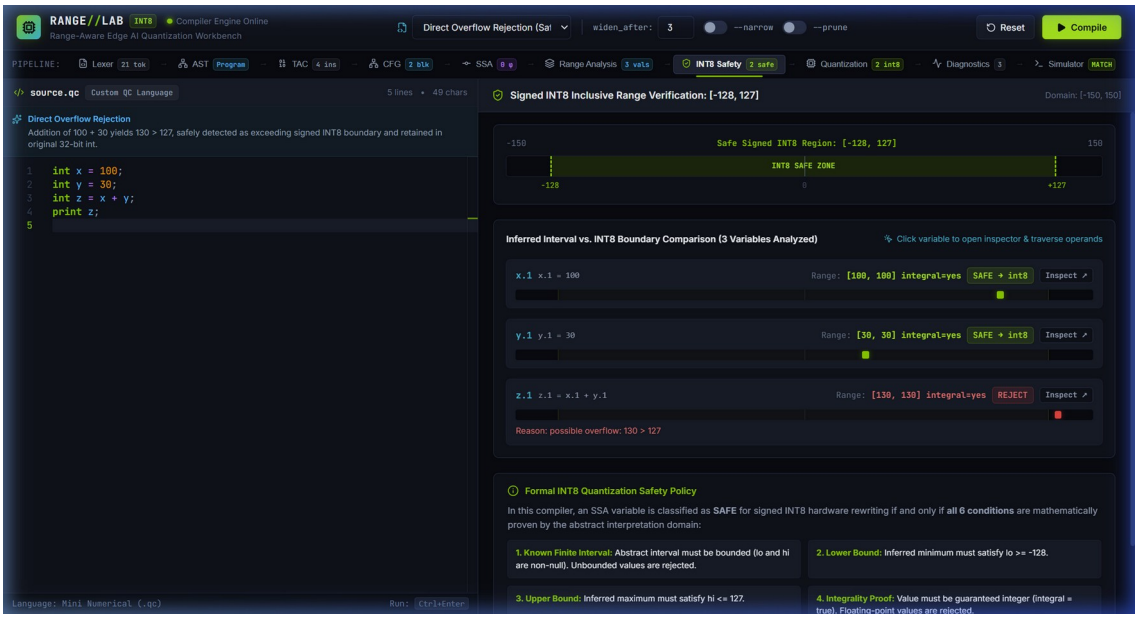


Figure 7.2: Direct overflow rejection for overflow_rejection.qc (x = 100, y = 30, z = x + y): z.1 = [130, 130] lies beyond the INT8 boundary and is marked REJECT with the reason “possible overflow: 130 > 127”, while x.1 and y.1 are SAFE → int8.

Evidence file: 03-overflow-rejection.png

Non-integral values. In `non_integral_rejection.qc` the values 10.0 and 3.5 are float constants, which are treated as non-integral. The sum 13.5 lies numerically inside [-128, 127], but it is rejected because representing it as an 8-bit integer would destroy its fractional part. In the benchmark all three variables of this program are rejected. This differs from the Review 2 example output, in which the constant 10.0 had been classified integral and safe; in the compiler of this report a float-typed constant is never integral, which is the more conservative behaviour (Section 7.6).

Unsupported operations. Division results are treated as non-integral, and comparisons are not rewritten by the quantization pass. In `unsupported_division.qc` (Appendix A, Program 7) the operands a and b are quantized to `int8`, while the quotient keeps its original `fp32` type: by the division rule of Table 5.1 its interval is [5, 5], but it is non-integral and its operation is unsupported. The benchmark `unsupported_op_rejection` uses the same program structure with the operands 20 and 4. In the loop and branch examples the relational temporaries (`t1` in Figures 5.2 and 7.3) are rejected because the relational comparison is not supported by the quantization pass.

7.3 Branch Refinement and Unreachable Code

The program `branch_refine.qc` sets `x = 15` and branches on `x >= 10` (Appendix A, Program 5). On the true edge the analysis refines x to [15, 15] ⊓ [10, +∞] = [15, 15] and computes `y.3 = x.1 - 5` = [10, 10]. On the false edge the constraint x < 10 intersects [15, 15]

in the empty set, so the edge is marked unreachable and `y.2 = x.1 + 5` is labelled UNREACHABLE and rejected with the reason “unreachable or infeasible control-flow path”. The relational temporary `t1 = x.1 >= 10` has interval [1, 1] (always true) and is rejected as an unsupported comparison. Figure 7.3 shows the result, which the analysis reaches in two sweeps.

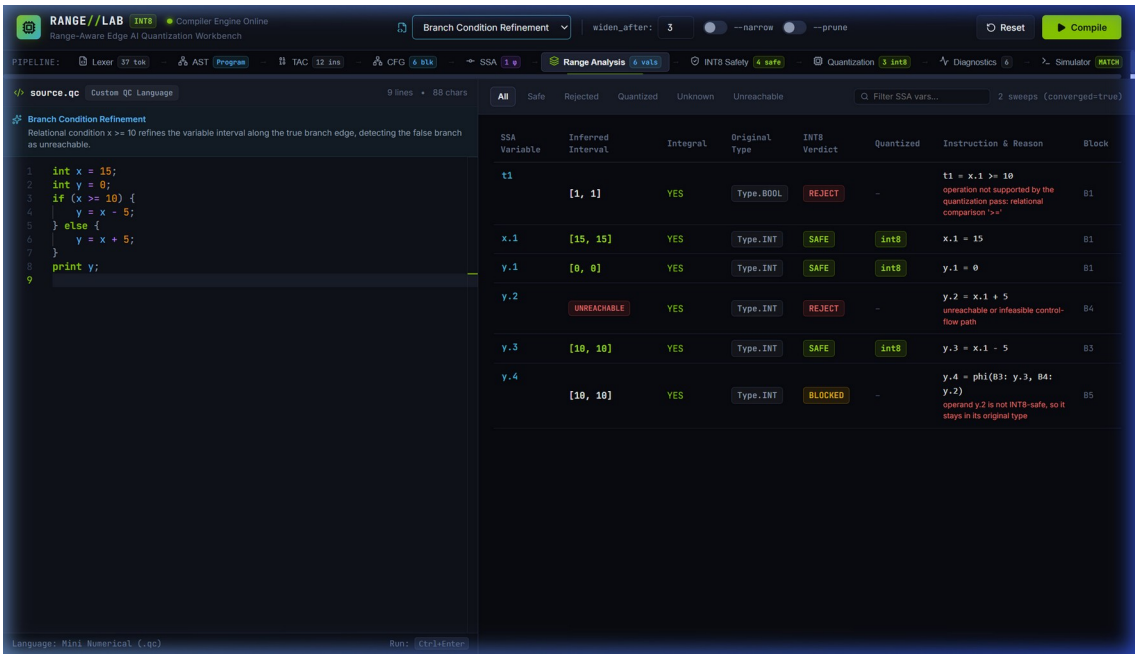


Figure 7.3: Path-sensitive branch refinement for branch_refine.qc (x = 15; if (x >= 10) y = x - 5 else y = x + 5): y.3 = [10, 10] is SAFE → int8, y.2 is marked UNREACHABLE, and the merge y.4 = phi(B3: y.3, B4: y.2) is BLOCKED.

Evidence file: 12-branch-refinement.png

The merge value `y.4` has the interval [10, 10], but the workbench labels it BLOCKED: the quantizer counts every incoming operand of a φ-node, and y.2, although unreachable, is not rewritten (Section 5.1.8). This is a conservative outcome. It costs precision, because the only reachable operand of y.4 is safe, but it cannot cause an unsafe conversion. It accounts for one of the three blocked variables in Table 7.1. Without refinement, the analysis could not have established that y.2 is dead, and the program would have been analysed as if both branches could execute.

7.4 Operand-Dependency Blocking

The benchmark `intermediate_overflow` (Section 5.1.8 and Figure 5.5) analyses four variables: a.1 and b.1 are rewritten to INT8, c.1 = 150 is rejected for overflow, and d.1 = c.1 - 50 evaluates to [100, 100] but is blocked because its operand c.1 is not rewritten. The result is 2 INT8, 1 blocked and 1 rejected variable (Table 7.1). Without the operand check, d.1 would have been rewritten after an upstream overflow. Together with the φ-node y.4 of Figure 7.3 and the φ-node max.4 of `if_else_phi`, d.1 accounts for the three blocked variables recorded in the evaluation; in each case the compiler reports the reason that an operand is not INT8-safe.

7.5 Theoretical Storage-Width Reduction

The only memory-related figure reported by this project is a theoretical static calculation of representation width. It assumes that every analysed SSA variable, including comparison temporaries, is stored in 32 bits in the baseline program and that a variable rewritten to INT8 is

stored in 8 bits, and it weights every variable equally, without regard to how often it is executed or how long it lives. Table 7.2 gives the derivation.

Table 7.2: Derivation of the theoretical representation-width reduction

Quantity	Value
Analysed SSA variables	54
Baseline width (54 × 32 bits)	1,728 bits
Variables rewritten to INT8	32
Bits saved (32 × (32 − 8))	768 bits
Variables kept at 32 bits (3 blocked + 19 rejected)	22
Width after quantization (32 × 8 + 22 × 32 = 256 + 704)	960 bits
Theoretical width reduction (768 / 1,728)	**44.44%**

$$\text{Reduction} = 32 \times (32 - 8) \;/\; (54 \times 32) \times 100\% = 768 \;/\; 1{,}728 \times 100\% \approx 44.44\% \tag{7.1}$$

> **Boundary of this result.** The 44.44% figure is a theoretical static representation-width estimate over the SSA variables of the 12 benchmarks. It is not a measurement of memory use, memory bandwidth, execution time, latency, energy or any other property of physical hardware, and it must not be read as a speed-up.

7.6 Comparison with the Review 2 Baseline

Table 7.3: Review 2 baseline compared with the Review 3 implementation

Dimension	Review 2 baseline	Review 3 implementation
Branch conditions	Flow-insensitive; conditions did not refine operand intervals	Path-sensitive refinement on true and false edges with unreachable-path detection
Loop analysis	Widening only; loop counters became UNKNOWN and were rejected	Widening followed by narrowing; bounded counters recover [0, 10]
Safety rules	Each value decided from its own interval	Per-value safety conditions plus an operand-dependency check in the quantizer, which also covers φ-nodes
Float constants	Example output classified 10.0 as integral and safe (1 safe, 2 rejected)	Float values treated as non-integral; all 3 variables rejected
Automated tests	20 tests, 0.018 s	141 tests, all passing
Evaluation	Manual runs of individual example programs	Reproducible harness over 12 benchmarks and 54 SSA variables
User interface	Command line only	RANGE//LAB workbench with ten stage views and an SSA inspector
Back-end integration	Standalone Python module calls	FastAPI server with a JSON serialiser

7.7 Discussion

Simulation equivalence. All 12 benchmarks produced matching output from the original and the quantized program (100.0% equivalence). This is empirical validation, on the executed

inputs, that 8-bit two's-complement arithmetic reproduces the 32-bit behaviour of the values the compiler chose to quantize. It is not a universal proof of equivalence over all inputs.

Safety. No safety violations were observed across the 12 evaluated benchmarks (0 violations over 54 SSA variables). This is an empirical finding that the safety rules behaved conservatively on the tested workloads; it is not, by itself, a formal proof of the correctness of the compiler for all programs.

Cost of conservatism. Twenty-two of the 54 variables (40.7%) were kept in 32-bit form. Table 7.4 classifies them by the primary reason recorded by the compiler. Several are conservative by design rather than genuinely unsafe: the five relational temporaries and the division result are rejected only because the pass does not rewrite those operations, and a φ-node or an expression is blocked when one of its operands is not rewritten. This reflects the accepted trade-off of static analysis, in which soundness is preserved at the expense of precision.

Table 7.4: Variables kept in their original representation, by primary reason (12 benchmarks)

Reason	Count	Variables
Unsupported operation	6	Comparison temporaries t1 in if_else_phi, while_loop_widening, bounded_loop_counter, stabilizing_loop and branch_refinement; the division result in unsupported_op_rejection
Unknown range	8	Loop-carried values in while_loop_widening (4) and stabilizing_loop (2), which are not narrowed in those runs; values on statically unreachable paths in if_else_phi and branch_refinement (2)
Possible overflow	2	z.1 in overflow_rejection; c.1 in intermediate_overflow
Non-integral value	3	x.1, y.1 and z.1 in non_integral_rejection
Blocked by an operand	3	max.4 in if_else_phi; y.4 in branch_refinement; d.1 in intermediate_overflow
Total	**22**	**19 rejected and 3 blocked**

Value of the Review 3 extensions. Narrowing turns the loop counter of the bounded-loop program from one quantized variable (4 / 1 / 1 / 0 / 3 without `--narrow`) into three (4 / 3 / 3 / 0 / 1 with it; Figure 5.3); refinement identifies dead code that the Review 2 analysis could not (Figure 7.3); and operand-dependency blocking prevents a value that is in range from being rewritten after an upstream overflow (Figure 5.5).

Threats to validity. The benchmarks are few, small and hand-written; the abstract domain is non-relational and therefore cannot represent relations such as $x - y \leq k$; and the results come from a software simulator. The results demonstrate the behaviour of the compiler on these programs and do not support any claim about run-time speed, latency, energy or deployment on edge hardware.

8. CONCLUSION AND FUTURE ENHANCEMENTS

8.1 Conclusion

This project set out to show that classical compiler techniques can decide, before execution, which values of a numerical program may safely be represented in INT8. It delivers a complete teaching-scale compiler for a small numerical language that parses and checks programs,

lowers them to TAC, builds a CFG and minimal SSA form, runs an interval-and-integrality abstract interpretation with path-sensitive refinement, widening and narrowing, applies a conservative INT8 safety rule together with an operand-dependency check, rewrites only the approved instructions, and validates the result with a software simulator. On the 12 benchmarks, 32 of 54 SSA variables (59.3%) were rewritten to INT8, 22 were conservatively kept in 32-bit form, all programs produced matching output after quantization, and no safety violations were observed. The theoretical static width reduction for these benchmarks is 44.44%. The 141 automated tests pass, and the RANGE//LAB workbench makes every stage and decision inspectable.

The main lesson is that the precision of the analysis, and not only its soundness, decides how much can be quantized: branch refinement and loop narrowing are what allow loop counters and conditional merges to qualify. The project also shows the value of separating the decision from the transformation, which made each part independently testable and every decision explainable by source location.

8.2 Implemented Contributions

Table 8.1: Features implemented and verified in this project

Feature	Evidence	Status
Mini numerical language and ten-stage compiler	Sections 4 and 5; Table 5.4	Implemented
SSA construction with dominance frontiers	Figure 4.4; test_ssa.py	Implemented
Exact-rational interval and integrality analysis	Table 5.1; test_range_analysis.py	Implemented
Path-sensitive branch refinement, unreachable-path detection	Figure 7.3; test_branch_refinement.py	Implemented
Widening and narrowing for loops	Figures 5.2, 5.3; test_loop_narrowing.py	Implemented
INT8 safety rules, operand-dependency blocking, selective retyping	Figures 5.4, 5.5, 7.2	Implemented
Software simulator with equivalence check	Figure 5.7; 12 of 12 benchmarks match	Implemented
FastAPI server and RANGE//LAB workbench	Figures 6.1–6.3; build and lint with 0 errors	Implemented
Automated tests and benchmark harness	141 tests (Table 5.5); 12 benchmarks (Figure 5.6)	Implemented

8.3 Known Limitations

- **Scalar arithmetic only.** The analysis handles scalar `int` and `float` variables; it does not support arrays, pointers, structures or dynamic allocation.
- **Non-relational domain.** Intervals are tracked independently per variable; relational facts such as $x - y \leq k$ cannot be represented.
- **Unbounded loops stay unknown.** Loops without a bounding exit condition, such as an unbounded accumulator, widen to $[-\infty, +\infty]$ and are soundly kept in 32-bit form.
- **Division is never quantized.** Division results are treated as non-integral, and comparisons are not rewritten by the quantization pass.

- **Conservative blocking.** A φ-node or expression with an operand that is not rewritten, including an operand on a statically unreachable path, is blocked even when its own interval fits (Figure 7.3), which costs precision.
- **Optional pruning mode.** The command-line option `--prune-unreachable` (the `--prune` toggle) was not used in any reported experiment. An imprecision found in it was corrected and covered by regression tests (Section 5.2.8); the mode has been compared with the default on the 12 benchmarks and 9 example files, but it is not part of the benchmark evaluation and is not claimed to be fully validated.
- **Dead branch followed by another conditional.** In the default mode, a statically dead branch followed by a further conditional jump, for example inside a loop, can make live code appear UNREACHABLE and give a loop-header interval narrower than the values it takes (reproducer in Section 5.2.8). No benchmark exhibits this, and the affected values were rejected or blocked in the cases examined, but it is not covered by the tests; applying the explicit dead-block tracking of the pruning mode to the default mode is future work.
- **No hardware code generation.** The back end ends in an interpreted simulator; it emits no ARM Cortex-M or RISC-V assembly and no object files, and no run-time, latency or energy measurements were made.
- **Wrap-around not directly tested.** The simulator's INT8 wrap-around has no dedicated test, because the safety pass never admits a value that would wrap.

8.4 Future Enhancements (not implemented)

The following directions are proposals only. None of them has been implemented, and none is claimed as a result of this project.

Table 8.2: Proposed future enhancements

Enhancement	Description	Status
Multi-dimensional array range analysis	Extend interval analysis to array index expressions in order to eliminate run-time bounds checks.	Not implemented
Affine loop unrolling and polyhedral scheduling	Unroll and tile loops before range analysis to expose more scalar quantization opportunities.	Not implemented
Fixed-point quantization synthesis	Synthesise scale factors S and zero-points Z for non-integral values, q = round(x / S) + Z.	Not implemented
Native LLVM IR generation	Emit target-independent LLVM IR with i8 types and compile to bare-metal microcontrollers with Clang/LLVM.	Not implemented
Measurement on edge hardware	Measure run-time, latency and energy of quantized code on a physical device, which this project has not done.	Not implemented
Relational abstract domains	Use octagons or polyhedra to retain relations between variables and recover precision.	Not implemented
Quantization of comparisons	Rewrite relational temporaries so that they no longer remain 32-bit.	Not implemented

9. REFERENCES

[1] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1738–1762, Aug. 2019.

[2] J. Chen and X. Ran, "Deep learning with edge computing: A review," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1655–1674, Aug. 2019.

[3] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.

[4] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: A whitepaper," *arXiv preprint arXiv:1806.08342 [cs.LG]*, Jun. 2018.

[5] P. Cousot and R. Cousot, "Static determination of dynamic properties of programs," in *Proc. 2nd Int. Symp. on Programming (ISOP)*, Paris, France: Dunod, 1976, pp. 106–130.

[6] P. Cousot and R. Cousot, "Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints," in *Proc. 4th ACM SIGACT-SIGPLAN Symp. on Principles of Programming Languages (POPL)*, 1977, pp. 238–252.

[7] F. Bourdoncle, "Efficient chaotic iteration strategies with widenings," in *Formal Methods in Programming and Their Applications*, LNCS vol. 735. Berlin, Germany: Springer, 1993, pp. 128–141.

[8] B. Blanchet, P. Cousot, R. Cousot, J. Feret, L. Mauborgne, A. Miné, D. Monniaux, and X. Rival, "A static analyzer for large safety-critical software," in *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2003, pp. 196–207.

[9] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck, "Efficiently computing static single assignment form and the control dependence graph," *ACM Trans. Program. Lang. Syst.*, vol. 13, no. 4, pp. 451–490, Oct. 1991.

[10] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis and transformation," in *Proc. Int. Symp. on Code Generation and Optimization (CGO)*, 2004, pp. 75–86.

[11] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe, "Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines," in *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2013, pp. 519–530.

[12] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, M. Cowan, H. Shen, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, "TVM: An automated end-to-end optimizing compiler for deep learning," in *Proc. 13th USENIX Symp. on Operating Systems Design and Implementation (OSDI)*, 2018, pp. 578–594.

APPENDIX A – SAMPLE CODE

This appendix lists seven sample `.qc` programs from the `examples/` directory of the project repository, reproduced as they appear in the files, together with the result the compiler produces for each. The compiler source code is held in the repository and is not reproduced here. All seven files exist under the names shown. The evaluation benchmarks of Section 7 are defined by source text embedded in `tests/evaluation.py`, not by the example files, so the relationship between the two was checked by comparing the program texts. Table A.1 reports the outcome of that comparison.

Table A.1: Sample programs and their relationship to the evaluation benchmarks

#	Example file	Benchmark row (Table 7.1)	Basis of the relationship
1	`examples/safe_arithmetic.qc`	`safe_arithmetic`	Program text identical
2	`examples/overflow_rejection.qc`	`overflow_rejection`	Program text identical
3	`examples/non_integral_rejection.qc`	`non_integral_rejection`	Program text identical
4	`examples/if_else_phi.qc`	`if_else_phi`	Program text identical
5	`examples/branch_refine.qc`	`branch_refinement`	Program text identical
6	`examples/bounded_loop.qc`	`bounded_loop_counter`	Program text identical; the benchmark runs with `--narrow`
7	`examples/unsupported_division.qc`	(none)	Not identical: the benchmark `unsupported_op_rejection` uses the operands 20 and 4, the example uses 40 and 8

Of the other example files, `while_loop.qc` is identical to the benchmark `while_loop_widening`, while `stabilizing_loop.qc` differs from the benchmark `stabilizing_loop`. The benchmarks `intermediate_overflow`, `boundary_fit_exact` and `edge_linear_transform` have no example file; the program of `intermediate_overflow` is listed in Section 5.1.8.

Program 1: Provably safe arithmetic (examples/safe_arithmetic.qc)

```
int x = 10;
int y = 20;
int z = x + y;
print z;
```

The analysis infers x.1 = [10, 10], y.1 = [20, 20] and z.1 = [30, 30]. All three are integral and inside [-128, 127], so all are rewritten to `int8` (Figures 5.4 and 5.7).

Program 2: Direct overflow rejection (examples/overflow_rejection.qc)

```
int x = 100;
int y = 30;
int z = x + y;
print z;
```

The analysis infers z.1 = [130, 130]. Since 130 > 127, z.1 is rejected and keeps its original `int` type, while x.1 and y.1 are rewritten (Figure 7.2). Both runs of the simulator print 130.

Program 3: Non-integral float rejection (examples/non_integral_rejection.qc)

```
float x = 10.0;
float y = 3.5;
float z = x + y;
print z;
```

Float-typed values are never integral. The sum z.1 = [13.5, 13.5] is numerically inside [-128, 127] but is rejected because it is not integral, which preserves the fractional part. All three variables are rejected in the benchmark.

Program 4: Control flow and φ-node merging (examples/if_else_phi.qc)

```
int x = 10;
int y = 20;
int max = 0;

if (x > y) {
    max = x;
} else {
    max = y;
}

print max;
```

The CFG has six blocks, B0 to B5, and SSA construction places one φ-node, `max.4 = phi(B4: max.2, B3: max.3)` (Figures 4.3 and 4.4). The compiler infers t1 = [0, 0] for the comparison, marks max.3 on the true path UNREACHABLE and gives max.4 the interval [20, 20]. The benchmark records 7 SSA variables, of which 4 are rewritten, 1 is blocked (max.4, because max.3 is not rewritten) and 2 are rejected (t1, an unsupported comparison, and max.3). Both runs print 20.

Program 5: Path-sensitive branch refinement (examples/branch_refine.qc)

```
int x = 15;
int y = 0;

if (x >= 10) {
    y = x - 5;
} else {
    y = x + 5;
}

print y;
```

Refinement narrows the true branch to y.3 = [10, 10]. On the false edge the constraint x < 10 intersects [15, 15] in the empty set, so y.2 is UNREACHABLE and rejected, and the merge y.4 = [10, 10] is blocked because y.2 is not rewritten (Figure 7.3).

Program 6: Bounded loop with narrowing (examples/bounded_loop.qc)

```
int i = 0;
while (i < 10) {
    i = i + 1;
}
print i;
```

Without `--narrow` the loop-header value is widened to UNKNOWN and only i.1 is rewritten. With `--narrow` the compiler recovers i.2 = [0, 10] and i.3 = [1, 10], and all three loop variables are rewritten to `int8` (Figures 5.2 and 5.3).

Program 7: Unsupported operation (examples/unsupported_division.qc)

```
// Unsupported operation example: real division is non-integral and
// unsupported by the INT8 quantization pass. Independent safe variables
// (a, b) are quantized to int8, while c is safely kept as fp32.
int a = 40;
int b = 8;
float c = a / b;
print c;
```

The operands a.1 = [40, 40] and b.1 = [8, 8] are rewritten to `int8`. The quotient c.1 = [5, 5] is flagged non-integral and its operation is unsupported, so it stays `fp32`.

Commands used with the sample programs

```
python -m src.main examples/safe_arithmetic.qc --emit ranges quantized simulate
python -m src.main examples/bounded_loop.qc --narrow --emit ranges simulate
python -m unittest discover -s tests -p "test_*.py" -v
python evaluate.py
```